UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica

APPENDICE

Esempi analisi LLM, modello GPT-4o

Anno Accademico 2025/2026

Indice

1 Esempi analisi LLM esercitazione 1 1
1.1 Esempi di risultati LLM nell'analisi di crash . . 1
1.2 Esempi di risultati LLM nell'analisi di timeout 1
1.3 Esempi di risultati LLM nell'analisi di IPC leak 4
1.4 Esempi di risultati LLM nell'analisi di dynamic failure 6
1.5 Esempi di risultati LLM nell'analisi di static failure 10
1.6 Esempi di risultati LLM nell'analisi di correct . 15
2 Esempi analisi LLM esercitazione 2 18
2.1 Esempi di risultati LLM nell'analisi di crash . . 18
2.2 Esempi di risultati LLM nell'analisi di timeout 20
2.3 Esempi di risultati LLM nell'analisi di IPC leak 23
2.4 Esempi di risultati LLM nell'analisi di dynamic failure 24
2.5 Esempi di risultati LLM nell'analisi di static failure 28
2.6 Esempi di risultati LLM nell'analisi di correct . 32
3 Esempi analisi LLM esercitazione 3 35
3.1 Esempi di risultati LLM nell'analisi di crash . . 35
3.2 Esempi di risultati LLM nell'analisi di timeout 38
3.3 Esempi di risultati LLM nell'analisi di dynamic failure 40
3.4 Esempi di risultati LLM nell'analisi di static failure 44
3.5 Esempi di risultati LLM nell'analisi di correct . 47
4 Esempi analisi LLM esercitazione 4 51
4.1 Esempi di risultati LLM nell'analisi di timeout 51
4.2 Esempi di risultati LLM nell'analisi di IPC leak 53
4.3 Esempi di risultati LLM nell'analisi di dynamic failure 56
4.4 Esempi di risultati LLM nell'analisi di static failure 60
4.5 Esempi di risultati LLM nell'analisi di correct . 63

5 Esempi analisi LLM esercitazione 5 67
5.1 Esempi di risultati LLM nell'analisi di timeout 67
5.2 Esempi di risultati LLM nell'analisi di IPC leak 69
5.3 Esempi di risultati LLM nell'analisi di dynamic failure 71
5.4 Esempi di risultati LLM nell'analisi di static failure 75
5.5 Esempi di risultati LLM nell'analisi di correct . 77

1 Esempi analisi LLM esercitazione 1

1.1 Esempi di risultati LLM nell'analisi di crash

Esempio crash

Studente: "daniicim"
Esercizio: "calcolo_parallelo_su_un_vettore_condiviso"
Commit: "fb66a8ea0844f5a5d9e5cff24dd5d3cb64a93275"
Test feedback: "Esercizio calcolo parallelo su vettore condiviso, con mutua esclusione. L'esercizio non è ancora stato svolto correttamente. L'esecuzione del programma è andata in crash"
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: processi-mutua-esclusione.c, function inizializza_semafori at line 26, the semctl call incorrectly specifies the semaphore number as 1 instead of 0. Since the semaphore set contains only one semaphore (created with semget at line 14), the valid semaphore number is 0. This causes undefined behavior, likely leading to the observed crash. The incorrect semaphore initialization prevents proper mutual exclusion, violating the synchronization policy required for the shared buffer."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the concrete issue in the diff: the semctl call in the student's code uses an invalid semaphore number (1 instead of 0), which is consistent with the observed crash and the semaphore set containing only one semaphore. The diagnosis aligns with the visible structural difference and the observed failure category."

1.2 Esempi di risultati LLM nell'analisi di timeout

Esempio timeout 1

Studente: "MickFury10"
Esercizio: "coppia_di_buffer"
Commit: "4c74ba1a44caab4590d6679cdeb188b3749ac424"
Test feedback: "Esercizio coppia di buffer. L'esercizio non è ancora stato svolto correttamente. L'esecuzione del programma

è andata in timeout"
LLM Code Correct: "NO"
LLM Code Diagnosis: "In main-consumatore.c, line 26, the shared memory for the second buffer is acquired using the same key as the first buffer (ftok(".", 'a') instead of ftok(".", 'b')). This causes both buf1_id and buf2_id to reference the same shared memory segment, leading to the "No such file or directory" error for buf2 during runtime. The correct key for buf2 should match the one used in main-padre.c for its creation (ftok(".", 'b')). This mismatch violates the shared memory allocation and access protocol."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the issue in main-consumatore.c at line 26, where the same key (ftok(".", 'a')) is used for both buf1_id and buf2_id, causing them to reference the same shared memory segment. This is consistent with the visible diff, which shows the student code using ftok(".", 'a') for buf2_id instead of the correct ftok(".", 'b') as in the solution. The diagnosis is specific, matches the observed failure (timeout due to incorrect shared memory access), and aligns with the visible difference in the diff."

Esempio timeout 2

Studente: "GiuseppeFus"
Esercizio: "simulazione_di_un_disco_con_un_vettore_circolare"
Commit: "7ef575eac8a54c6e198b21ddf6a4a4be25eca875"
Test feedback: "Esercizio simulazione disco, con semafori. L'esercizio non è ancora stato svolto correttamente. L'esecuzione del programma è andata in timeout"
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: coda_richieste.c, function inizializza_coda (lines 25-27), the semaphore SPAZIO_DISP is incorrectly initialized using semctl(sem_id, SPAZIO_DISP, DIM) instead of semctl(sem_id, SPAZIO_DISP, SETVAL, DIM). This results in an invalid semaphore initialization, which can cause the semaphore operations to fail or block indefinitely.

This defect likely leads to the observed timeout during execution, as processes may wait indefinitely for semaphore signals that are not properly configured."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a concrete structural difference in the initialization of the SPAZIO_DISP semaphore, where the student code uses semctl(sem_id, SPAZIO_DISP, DIM) instead of the correct semctl(sem_id, SPAZIO_DISP, SETVAL, DIM) as shown in the solution. This difference is consistent with the observed timeout failure, as improper semaphore initialization can lead to indefinite blocking. The diagnosis does not introduce unrelated or speculative issues and aligns with the visible diff and feedback."

Esempio timeout 3

Studente: "annxj"
Esercizio: "simulazione_di_un_disco_con_un_vettore _circolare"
Commit: "c3d56aa36d399d951e63c9494176b41564e40e50"
Test feedback: "Esercizio simulazione disco, con semafori. L'esercizio non è ancora stato svolto correttamente. L'esecuzione del programma è andata in timeout"
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: coda_richieste.c, function inserisci_richiesta (line 97), the assignment c->vettore[c->testa].posizione = r->processo overwrites the previous assignment to c->vettore[c->testa].posizione (line 97). This results in the posizione field being incorrectly set to the process ID instead of the intended disk position. This violates the protocol for correctly storing requests in the circular queue and likely causes the observed timeout due to incorrect data being processed by the Schedulatore."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies that the assignment c->vettore[c->testa].posizione = r->processo overwrites the previous assignment to c->vettore[c->testa].posizione = r->posizione in the student's code. This is

visible in the diff and matches the observed timeout failure, as the incorrect data assignment likely disrupts the protocol for storing requests in the circular queue. The diagnosis is specific, consistent with the diff, and does not introduce unrelated or speculative issues."

1.3 Esempi di risultati LLM nell'analisi di IPC leak

Esempio IPC leak 1

Studente: "GiuseppeGuarino"
Esercizio: "lettori-scrittori_con_semafori_su _una_coppia_di_valori_condivisa"
Commit: "0412a6e0f1b341c8c38d0cdd420bcd84718b52b0"
Test feedback: "Esercizio lettori-scrittori. L'esercizio non è ancora stato svolto correttamente. È necessario deallocare le risorse IPC al termine della esecuzione"
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: main_padre.c, lines 54-61, the parent process creates five writer processes instead of one, violating the exercise requirement of a single writer process. This leads to multiple writers competing for the shared buffer, which can cause synchronization issues and resource leaks, as observed in the runtime output with ipc_leak symptoms. The defect directly contradicts the exercise's explicit requirement for one writer and two readers."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies that the student code creates multiple writer processes instead of a single one, as required by the exercise. This is supported by the diff in main_padre.c, where the student code lacks the proper logic to ensure only one writer process is created. The feedback mentioning IPC resource leaks aligns with the synchronization issues caused by multiple writers. The diagnosis is specific, concrete, and supported by the diff."

Esempio IPC leak 2

Studente: “al3ssandr0rabb0ni”
Esercizio: “simulazione_di_un_disco_con_un_vettore_circolare”
Commit: “d5f16c05db479ce8fbac884dc5c591b65f61341b”
Test feedback: “Esercizio simulazione disco, con semafori. L’esercizio non è ancora stato svolto correttamente. È necessario deallocare le risorse IPC al termine della esecuzione”
LLM Code Correct: “NO”
LLM Code Diagnosis: “The defect lies in FILE: coda_richieste.c, function inizializza_coda() at lines 51-52. The sem_id and shm_id fields of the coda_richieste structure are incorrectly initialized to 0 instead of being assigned the actual semaphore and shared memory IDs created earlier (sem_id and shm_id variables). This causes invalid semaphore operations (Errore WAIT: Invalid argument and Errore SIGNAL: Invalid argument) during runtime, as the semaphore ID used in Wait_Sem and Signal_Sem is incorrect. This violates the ownership/lifetime invariant for shared resources.”
Judge Code Correct: “YES”
Judge Code Motivation: “The diagnosis correctly identifies that the sem_id and shm_id fields in the coda_richieste structure are initialized to 0 instead of being assigned the actual semaphore and shared memory IDs (sem_id and shm_id variables). This is explicitly visible in the diff (lines 51-52 in the student code vs. lines 48-49 in the solution). The observed runtime errors related to invalid semaphore operations are consistent with this defect. The diagnosis does not introduce unrelated or speculative issues and matches the concrete mechanism visible in the diff.”

Esempio IPC leak 3

Studente: “mickdicostanzo”
Esercizio: “calcolo_parallelo_su_un_vettore_condiviso”
Commit: “2b57404f8333f76197c4ec6cbc3f2c55ec7f5d80”
Test feedback: “Esercizio calcolo parallelo su vettore condiviso, con mutua esclusione. L’esercizio non è ancora stato svolto correttamente. È necessario deallocare le risorse IPC al termine della esecuzione”

LLM Code Correct: "NO"
LLM Code Diagnosis: "In main.c, the buffer_key used in shmget at line 54 is uninitialized, leading to undefined behavior when creating the shared memory segment for the buffer. This can cause an IPC leak as the key may conflict with existing resources or fail to allocate properly. The key should be explicitly initialized, similar to key_vett at line 23, to ensure proper shared memory creation and avoid resource mismanagement."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies that buffer_key is uninitialized in the student's code at line 54, while the solution uses IPC_PRIVATE directly in shmget. This aligns with the feedback about potential IPC resource mismanagement due to improper initialization. The diff supports the claimed defect mechanism."

1.4 Esempi di risultati LLM nell'analisi di dynamic failure

Esempio dynamic failure 1

Studente: "CiroCorvino1998"
Esercizio: "lettori-scrittori_con_semafori_su_una_coppia_di_valori_condivisa"
Commit: "2496a98b15c59dec9275923af7fb26a794bb027b"
Test feedback: "Esercizio lettori-scrittori. L'esercizio non è ancora stato svolto correttamente. L'esecuzione non è corretta: La lettura del buffer non restituisce l ultima coppia di valori scritta"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output contains a mismatch in the protocol. The "Fine scrittura" messages are printed by the readers (PIDs 92001 and 92002) instead of the writer (PID 92000). This indicates a role identity issue, as "Fine scrittura" should only be printed by the writer. Additionally, the writer performs 5 writes, but the readers perform 10 reads each, which exceeds the expected 5 reads per reader. This violates the expected counts and pairing of operations."
LLM Code Correct: "NO"

LLM Code Diagnosis: "In FILE: buffer.c, function leggi_buffer, line 29, the reader process incorrectly prints the message "[Fine scrittura]", which is meant for the writer process. This indicates a role identity issue, as the reader should not print a message indicating the completion of a write operation. This defect directly explains the protocol mismatch observed in the runtime output, where readers print writer-specific messages, violating the exercise's role-specific behavior requirements."
Judge Output Correct: "NO"
Judge Output Motivation: "The diagnosis identifies specific issues (role identity problem and mismatched operation counts) that are not mentioned in the ground truth. The ground truth specifies a different failure: the buffer does not return the last written pair of values during reading. While both describe incorrect execution, the diagnosis does not align semantically with the ground truth's stated failure type."
Judge Code Correct: "NO"
Judge Code Motivation: "The diagnosis claims a role identity issue where the reader process incorrectly prints a writer-specific message ("[Fine scrittura]"). However, the diff does not show any such incorrect message being printed by the reader process. Instead, the reader's output in both the student and solution code is consistent with "[

Esempio dynamic failure 2

Studente: "giovanni158"
Esercizio: "simulazione_di_un_disco_con_un_vettore_circolare"
Commit: "05d67a7abf86a705bc7ef4854b0efc3878b3fc8b"
Test feedback: "Esercizio simulazione disco, con semafori. L'esercizio non è ancora stato svolto correttamente. L'esecuzione non è corretta: Il numero di richieste presenti nel vettore circolare esce dai limiti consentiti [0,10]. Il numero di processi che producono richieste non coincide con il numero di processi da cui il disco preleva richieste"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output shows multiple issues:

(1) All requests have the same position value (1), which contradicts the requirement for random positions between 0 and 19. (2) The "processo" field in all requests is 0, which is inconsistent with the expected use of the process PID. (3) The scheduler's "Attesa Schedulatore" times are all 0 seconds after the first request, which is incorrect as the time should depend on the absolute difference between consecutive positions. These issues indicate protocol and value mismatches."
LLM Code Correct: "NO"
LLM Code Diagnosis: "The function Utente in FILE: main.c (lines 39-51) fails to initialize the fields posizione and processo of the richiesta structure before calling inserisci_richiesta. This results in all requests having the same posizione value (1) and processo value (0), violating the requirement for random positions between 0 and 19 and the use of the process PID. This defect directly causes the observed runtime failure where the scheduler's wait times are incorrect and all requests are identical."
Judge Output Correct: "NO"
Judge Output Motivation: "The diagnosis identifies issues (e.g., repeated position values, incorrect "processo" field, and scheduler timing) that are unrelated to the ground truth failures, which focus on the circular vector's request count exceeding limits and mismatched producer-consumer processes. The diagnosis does not semantically align with the ground truth's described failures."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies that the student code fails to initialize the posizione and processo fields of the richiesta structure in the Utente function before calling inserisci_richiesta. This defect is explicitly visible in the diff, where the solution initializes these fields (r.processo = getpid(); and r.posizione = rand()

Esempio dynamic failure 3

Studente: "ManuelParasmo"
Esercizio: "simulazione_di_un_disco_con_un_vettore_circolare"

Commit: “abd301470a74bc559b5acf13fdfe98956cb099c9”
Test feedback: “Esercizio simulazione disco, con semafori. L’esercizio non è ancora stato svolto correttamente. L’esecuzione non è corretta: La produzione nel vettore circolare non avviene nella posizione di testa attesa. Il numero di processi che producono richieste non coincide con il numero di processi da cui il disco preleva richieste”
LLM Output Correct: “NO”
LLM Output Diagnosis: “The output shows mismatches in the protocol. Specifically, the scheduler processes 26 requests instead of the expected 25, violating the termination condition. Additionally, some requests have inconsistent or impossible values, such as "Prelevo richiesta: posizione=19, processo=158035, " which does not match any prior production. These issues indicate a failure in maintaining protocol coherence and value consistency.”
LLM Code Correct: “NO”
LLM Code Diagnosis: “The function Schedulatore in FILE: main.c processes 26 requests instead of the expected 25 due to an off-by-one error in the loop condition at line 19 (for (int i = 0; i $<$ 25; i++)). The loop should terminate after serving exactly 25 requests, but the scheduler processes an additional request, violating the termination condition. This defect directly explains the observed runtime failure where the scheduler consumes an extra request, leading to protocol mismatch and incorrect behavior.”
Judge Output Correct: “YES”
Judge Output Motivation: “The diagnosis aligns with the ground truth by identifying mismatches in the protocol, including incorrect request processing counts and value inconsistencies. These issues correspond to the ground truth’s mention of incorrect execution, mismatched production and consumption, and unexpected positions in the circular vector. The diagnosis provides a more detailed explanation of the same failures.”
Judge Code Correct: “NO”
Judge Code Motivation: “The diagnosis claims an off-by-one error in the loop condition of the Schedulatore function in main.c, specifically at line 19. However, the provided diff does not include any evidence of a loop in Schedulatore or any related off-by-one error. The cited mechanism is not concretely supported by the

visible differences, and the diagnosis appears speculative."

1.5 Esempi di risultati LLM nell'analisi di static failure

Esempio static failure 1

Studente: "alecsgal"
Esercizio: "coppia_di_buffer"
Commit: "ed548ccc7949548371e0e15e0545e405a079534d"
Test feedback: "Esercizio coppia di buffer. L'esercizio non è ancora stato svolto correttamente. Anche se il programma esegue, sono stati riscontrati i seguenti difetti all'interno del codice (main-consumatore.c)"
Static warnings: "["file": "main-consumatore.c", "line": 47, "message": "In questo esercizio, è necessario utilizzare ftok() per le chiavi delle risorse. I processi figli devono ottenere la stessa risorsa che è stata creata dal processo padre. Con IPC_PRIVATE, viene creata una nuova risorsa ad ogni operazione di shmget.", "file": "main-consumatore.c", "line": 66, "message": "In questo esercizio, è necessario utilizzare ftok() per le chiavi delle risorse. I processi figli devono ottenere la stessa risorsa che è stata creata dal processo padre. Con IPC_PRIVATE, viene creata una nuova risorsa ad ogni operazione di semget.", "file": "main-padre.c", "line": 48, "message": "In questo esercizio, è necessario utilizzare ftok() per le chiavi delle risorse. I processi figli devono ottenere la stessa risorsa che è stata creata dal processo padre. Con IPC_PRIVATE, viene creata una nuova risorsa ad ogni operazione di shmget.",, "file": "main-padre.c", "line": 88, "message": "In questo esercizio, è necessario utilizzare ftok() per le chiavi delle risorse. I processi figli devono ottenere la stessa risorsa che è stata creata dal processo padre. Con IPC_PRIVATE, viene creata una nuova risorsa ad ogni operazione di semget.", "file": "main-produttore.c", "line": 47, "message": "In questo esercizio, è necessario utilizzare ftok() per le chiavi delle risorse, per fare in modo che tutte le chiamate a shmget() ottengano la stessa risorsa condivisa. I processi figli devono ottenere la stessa risorsa che è stata creata dal processo padre. Con IPC_PRIVATE, viene

creata una nuova risorsa ad ogni operazione di shmget().", "file": "main-produttore.c", "line": 66, "message": "In questo esercizio, è necessario utilizzare ftok() per le chiavi delle risorse, per fare in modo che tutte le chiamate a semget() ottengano la stessa risorsa condivisa. I processi figli devono ottenere la stessa risorsa che è stata creata dal processo padre. Con IPC_PRIVATE, viene creata una nuova risorsa ad ogni operazione di semget()."]"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output shows a mismatch in the protocol. Specifically, the value 8 is produced twice consecutively by the producer without an intervening consumption of the first 8. This violates the expected producer-consumer protocol, where each produced value should be consumed before the same buffer is reused. Additionally, the final consumption of the second 8 does not align with the expected pairing and completion of 5 production-consumption cycles."
LLM Code Correct: "NO"
LLM Code Diagnosis: "FILE: buffer.c, function produzione, lines 24-35: The selector logic for choosing a buffer does not ensure that the selected buffer is in the LIBERO state. If both buffers are not LIBERO, the code defaults to using buf2 without verifying its state. This violates the exercise constraint that the producer must wait until at least one buffer is LIBERO before proceeding. The absence of a proper check for both buffers' states before selection is a structural defect in the selector logic."
Judge Output Correct: "NO"
Judge Output Motivation: "The ground truth states that the output is correct, but the diagnosis claims a specific protocol violation and mismatched behavior in the output. This contradicts the ground truth, which explicitly confirms correctness."
Judge Code Correct: "NO"
Judge Code Motivation: "The diagnosis identifies a defect in the selector logic for buffer state validation in the produzione function, which is unrelated to the ground truth warnings about the use of ftok() for resource keys and the avoidance of IPC_PRIVATE. The diagnosis and ground truth address entirely different defect families (selector logic vs. resource key usage)."

Esempio static failure 2

Studente: “daniicim”
Esercizio: “lettori-scrittori_con_semafori_su_una_coppia_di_valori_condivisa”
Commit: “89d20703b09ffe721d28821f1153a3f03de8a129”
Test feedback: “Esercizio lettori-scrittori. L’esercizio non è ancora stato svolto correttamente. Anche se il programma esegue, sono stati riscontrati i seguenti difetti all’interno del codice (buffer.c)”
Static warnings: “["file": "buffer.c", "line": 28, "message": "Il codice dei lettori non è corretto. Per svolgere l’esercizio, è necessario applicare l’algoritmo visto a lezione, basato sui semafori ’MUTEXL’ e ’SYNCH’, e la variabile condivisa ’NUM_LETTORI’.", "file": "main_padre.c", "line": 62, "message": "Creazione dei semafori errata. Per risolvere l’esercizio, è necessario creare i semafori richiesti dall’algoritmo visto a lezione."]”
LLM Output Correct: “NO”
LLM Output Diagnosis: “The output contains multiple "Errore WAIT: File too large" and "Errore SIGNAL: File too large" messages, indicating semaphore-related errors. These errors suggest that the synchronization mechanism is not functioning correctly. Additionally, the scrittore performs 5 writes, but the lettori read the same values multiple times, indicating a lack of proper synchronization between the writer and readers. This violates the protocol requiring proper synchronization to ensure consistent and unique reads for each write.”
LLM Code Correct: “NO”
LLM Code Diagnosis: “The synchronization policy for the reader-writer problem is violated in FILE: buffer.c, function leggi_buffer. The semaphore MUTEXL is used for both readers and writers, but it is released at line 24 without incrementing or tracking the num_lettori variable. This causes over-serialization of readers, preventing simultaneous reads, which contradicts the exercise requirement that readers should be able to read concurrently. The correct policy should increment num_lettori and use a separate semaphore to manage reader count.”
Judge Output Correct: “NO”
Judge Output Motivation: “The ground truth explicitly states

that the visible execution/output is correct, while the diagnosis claims multiple synchronization-related errors and protocol violations. This directly contradicts the ground truth, which confirms no issues in the output. The diagnosis invents problems that are not present in the ground truth."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the synchronization issue in the reader-writer problem, specifically the improper use of the MUTEXL semaphore and the failure to manage the NUM_LETTORI variable, which aligns with the ground truth warning about incorrect reader code and the need to apply the algorithm taught in class. The diagnosis addresses the same defect mechanism and provides a detailed explanation within the same defect family."

Esempio static failure 3

Studente: "emanueleascolese"
Esercizio: "coppia_di_buffer"
Commit: "7b635334227487db81d5efc576a962b49c1151d0"
Test feedback: "Esercizio coppia di buffer. L'esercizio non è ancora stato svolto correttamente. Anche se il programma esegue, sono stati riscontrati i seguenti difetti all'interno del codice (main-consumatore.c)"
Static warnings: "["file": "main-consumatore.c", "line": 45, "message": "Nei processi figli, non è necessario indicare i permessi in shmget(), poiché i permessi sono stati già assegnati al momento della creazione della risorsa da parte del processo padre. È sufficiente passare 0 come terzo parametro.", "file": "main-consumatore.c", "line": 53, "message": "Nei processi figli, non è necessario indicare i permessi in shmget(), poiché i permessi sono stati già assegnati al momento della creazione della risorsa da parte del processo padre. È sufficiente passare 0 come terzo parametro.", "file": "main-consumatore.c", "line": 81, "message": "Nei processi figli, non è necessario indicare i permessi in semget(), poiché i permessi sono stati già assegnati al momento della creazione della risorsa da parte del processo padre. È sufficiente passare 0 come terzo parametro."]"

LLM Output Correct: "YES"
LLM Output Diagnosis: "No concrete output mismatch is visible. The trace structure is consistent with the exercise protocol. The producer and consumer alternate operations correctly, with values being produced and consumed in a coherent sequence. All expected phases (production and consumption) are present, and the counts match the expected 5 productions and 5 consumptions. No pairing, ordering, or value errors are evident."
LLM Code Correct: "NO"
LLM Code Diagnosis: "FILE: buffer.c, function produzione, lines 26-37: The selector logic for choosing a "LIBERO" buffer does not ensure mutual exclusion when checking and updating the buffer state. Although the comment states that mutual exclusion is unnecessary due to a single producer, this assumption is incorrect because the producer may still face race conditions with the consumer, which can access and modify the buffer state concurrently. A synchronization mechanism, such as a semaphore or mutex, is required to ensure atomicity of these operations."
Judge Output Correct: "YES"
Judge Output Motivation: "The diagnosis correctly states that no concrete output mismatch is visible and aligns with the ground truth, which confirms the output is correct. It does not introduce any invented problems or contradictions."
Judge Code Correct: "NO"
Judge Code Motivation: "The diagnosis discusses a synchronization issue in buffer.c related to mutual exclusion, which is unrelated to the ground truth warnings. The ground truth warnings pertain to unnecessary permission parameters in shmget() and semget() calls in main-consumatore.c. These are entirely different defect families (synchronization vs. resource access permissions), and there is no causal or semantic overlap between them."

1.6 Esempi di risultati LLM nell'analisi di correct

Esempio correct 1

Studente: "MickFury10"
Esercizio: "lettori-scrittori_con_semafori_su_una_coppia_di_valori_condivisa"
Commit: "26743b3b58e845dff3c7ce0030a607cdcb1d3ce3"
Test feedback: "Esercizio lettori-scrittori.Congratulazioni, l'esercizio compila ed esegue correttamente, e non è stato riscontrato nessuno degli errori frequenti nel codice"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output shows errors in creating shared memory ("Errore creazione shared memory: No such file or directory") for multiple processes. This indicates a failure in the initialization phase, preventing the expected interactions (writing and reading) from occurring. Additionally, no concrete evidence of the required 5 writes and 5 reads is visible in the trace, and the program terminates prematurely with resource deallocation. The trace is incomplete and does not fulfill the exercise protocol."
LLM Code Correct: "NO"
LLM Code Diagnosis: "The shared memory key (key_t chiave_shm) used in main_lettori.c and main_scrittore.c is inconsistent with the key defined in main_padre.c. Specifically, main_lettori.c and main_scrittore.c use ftok(".", 't'), while main_padre.c uses ftok(".", 'r'). This mismatch causes the shared memory to not be found, as evidenced by the runtime error "Errore creazione shared memory: No such file or directory." This is a clear violation of the requirement to use a shared memory buffer for communication."
Judge Output Correct: "NO"
Judge Output Motivation: "The ground truth states that the visible execution/output is correct, while the diagnosis claims multiple errors, including shared memory creation failure, incomplete trace, and protocol non-fulfillment. These claims contradict the ground truth and invent problems not present in the output."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis claims a mismatch in shared memory keys (key_t chiave_shm) between

main_lettori.c, main_scrittore.c, and main_padre.c. The diff confirms that the student uses inconsistent keys ('f' and 't' in main_lettori.c and main_scrittore.c, and 'r' in main_padre.c), while the solution uses a consistent key ('m') across all files. This directly supports the claimed defect mechanism of shared memory not being found due to key inconsistency."

Esempio correct 2

Studente: "barigio"
Esercizio: "simulazione_di_un_disco_con_un_vettore_circolare"
Commit: "3fdca5ee55eb42ca1a7c7cb1c2d8e8cfa9e6df52"
Test feedback: "Esercizio simulazione disco, con semafori.Congratulazioni, l'esercizio compila ed esegue correttamente, e non è stato riscontrato nessuno degli errori frequenti nel codice"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output shows mismatches in the circular queue behavior. Specifically, the "Produzione in testa" indices do not follow a circular pattern within the queue size of 10. For example, indices 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 are followed by 0, 2, 5, 8, 1, etc., which skips expected values and does not wrap around correctly. This violates the circular queue protocol. Additionally, the total number of requests produced (25) matches the expected count, but the queue behavior is inconsistent with the exercise requirements."
LLM Code Correct: "YES"
LLM Code Diagnosis: "No concrete correctness concrete defect is visible from the provided code and runtime context. The program adheres to the explicit requirements of the exercise, including the use of semaphores for synchronization, proper handling of the circular queue, and correct scheduling of disk operations. The observed output indicates that all processes executed as expected, with no signs of incomplete protocol phases, missing termination messages, or structural issues. The shared memory and semaphore mechanisms are correctly initialized, used, and deallocated. The program successfully handles the production and consumption of requests without any observable errors or violations."

Judge Output Correct: "NO"
Judge Output Motivation: "The ground truth explicitly states that the visible execution/output is correct, while the diagnosis claims there are mismatches in the circular queue behavior and violations of the circular queue protocol. This directly contradicts the ground truth, which confirms the correctness of the output. The diagnosis invents a problem that does not exist according to the ground truth."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly states that the code is functionally correct, as the observed feedback explicitly confirms no errors or warnings. The diff does not show any explicit correctness violations, and the structural differences (e.g., semaphore naming, initialization order, or additional semaphores) do not constitute a correctness defect. The diagnosis aligns with the rules and observed feedback."

Esempio correct 3

Studente: "Moriarty2002"
Esercizio: "calcolo_parallelo_su_un_vettore_condiviso"
Commit: "8b1bd9db804054b252c33180b43178f732e036dd"
Test feedback: "Esercizio calcolo parallelo su vettore condiviso, con mutua esclusione.Congratulazioni, l'esercizio compila ed esegue correttamente, e non è stato riscontrato nessuno degli errori frequenti nel codice"
LLM Output Correct: "YES"
LLM Output Diagnosis: "The output is consistent with the exercise protocol. All 10 child processes are present, each working on a distinct portion of the vector. Each child reports its local minimum, and the parent correctly identifies the global minimum as 421686, which matches the smallest local minimum. No concrete output mismatches or structural issues are visible."
LLM Code Correct: "NO"
LLM Code Diagnosis: "In processi-mutua-esclusione.c, the padre function contains a loop on line 69 that uses NUM_ELEMENTI instead of NUM_PROCESSI to wait for child processes. This causes the parent process to call

wait(NULL) 10, 000 times instead of 10, leading to incorrect behavior and potential resource mismanagement."
Judge Output Correct: "YES"
Judge Output Motivation: "The diagnosis correctly states that the output is consistent with the exercise protocol and explicitly confirms that no concrete output mismatches or structural issues are visible, aligning with the ground truth that the visible execution/output is correct. No invented problems are added."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies that the loop in the padre function uses NUM_ELEMENTI instead of NUM_PROCESSI to call wait(NULL). This is directly supported by the diff, where the student code uses for (size_t i = 0; i < NUM_ELEMENTI; ++i) while the solution uses for (int i = 0; i < NUM_PROCESSI; i). The feedback does not contradict this, and the cited defect mechanism is visible in the diff."

2 Esempi analisi LLM esercitazione 2

2.1 Esempi di risultati LLM nell'analisi di crash

Esempio crash 1

Studente: "daniicim"
Esercizio: "produttore-consumatore_con_priorita"
Commit: "a7ab493d41bfc3a5f6493f43fe1e79d791d94573"
Test feedback: "Esercizio produttore-consumatore con priorita, con processi. L'esercizio non è ancora stato svolto correttamente. L'esecuzione del programma è andata in crash"
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: procedure.c, function produci_alta_prio (lines 31-63), the condition while(p->count == DIM || p->num_lettori_alta_priorita > 0) incorrectly includes p->num_lettori_alta_priorita in the buffer-full check. This variable tracks the number of high-priority producers waiting, not the buffer state. This causes high-priority producers to block unnecessarily, leading to a deadlock when no producer can proceed.

This explains the observed crash due to a timeout. The correct condition should only check p->count == DIM for buffer fullness, as high-priority producers should not block themselves based on their own waiting count."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a concrete structural issue in the student's code, specifically the incorrect inclusion of p->num_lettori_alta_priorita in the buffer-full condition, which is visible in the diff. This aligns with the observed crash due to a timeout, as the condition could lead to unnecessary blocking and deadlock. The diagnosis is consistent with the diff and observed failure, and no unrelated or speculative issues are mentioned."

Esempio crash 2

Studente: "Mashtemah"
Esercizio: "produttore-consumatore_con_priorita"
Commit: "03fd59b4bdda4dcaedaeeb89adb639d732c9c86f"
Test feedback: "Esercizio produttore-consumatore con priorita, con processi. L'esercizio non è ancora stato svolto correttamente. L'esecuzione del programma è andata in crash"
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: main.c, line 33, the shmget call uses an incorrect combination of flags (IPC_CREAT || 0644), which results in a logical OR instead of a bitwise OR. This causes the shared memory segment creation to fail, leading to a crash when subsequent operations attempt to access the shared memory. The correct flag combination should be IPC_CREAT | 0644. This defect directly explains the observed runtime failure (crash)."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the use of a logical OR (||) instead of a bitwise OR (|) in the shmget call on line 33 of main.c. This defect is explicitly visible in the diff and directly explains the observed crash due to the failure in shared memory segment creation. The diagnosis is specific, consistent with the diff, and matches the observed failure mechanism."

Esempio crash 3

Studente: “elenaoliviero14”
Esercizio: “simulazione_di_un_disco_con_un_vettore_circolare”
Commit: “7b4af500dc9596317897468f277b7165c7272b5b”
Test feedback: “Esercizio simulazione disco, con monitor. L’esercizio non è ancora stato svolto correttamente. L’esecuzione del programma è andata in crash”
LLM Code Correct: “NO”
LLM Code Diagnosis: “In main.c, line 120, the shared memory allocation for MonitorSchedulatore is incorrect. The shmget call is missing its first argument, the key, and the second argument, the size, is not properly passed. This results in an invalid shared memory segment creation, leading to a crash when attempting to attach or use the memory. The correct syntax should include a valid key (e.g., IPC_PRIVATE) and the size of the structure (sizeof(MonitorSchedulatore)). This defect directly causes the observed runtime failure.”
Judge Code Correct: “YES”
Judge Code Motivation: “The diagnosis correctly identifies the issue with the shmget call in main.c at line 120. The diff shows that the student code is missing the key argument and improperly constructs the shmget call, which aligns with the observed crash. The diagnosis is specific, matches the visible defect in the diff, and is consistent with the feedback.”

2.2 Esempi di risultati LLM nell’analisi di timeout

Esempio timeout 1

Studente: “GiuseppeFus”
Esercizio: “prelievi_multipli”
Commit: “c2dd773c9d9806a05ab3e6432e99fb3756b99384”
Test feedback: “Esercizio prelievi multipli. L’esercizio non è ancora stato svolto correttamente. L’esecuzione del programma è andata in timeout”
LLM Code Correct: “NO”

LLM Code Diagnosis: "In FILE: prodcons.c, function produci, the condition at line 62 incorrectly checks if(p->totale_elementi > DIM) instead of if(p->totale_elementi >= DIM). This allows a producer to proceed even when the buffer is full, violating the FIFO buffer constraint and potentially causing undefined behavior. This defect can lead to a deadlock or timeout, as observed in the runtime failure, because producers and consumers may block indefinitely due to inconsistent buffer state."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a concrete defect in the condition at line 62 of produci in prodcons.c, which is visible in the diff. The student code checks if(p->totale_elementi > DIM) instead of if(p->totale_elementi >= DIM), as claimed. This defect aligns with the described mechanism of allowing a producer to proceed when the buffer is full, potentially causing a timeout due to inconsistent buffer state. The diagnosis is specific, supported by the diff, and matches the observed runtime failure."

Esempio timeout 2

Studente: "LorenzoLubrano"
Esercizio: "produttore-consumatore_con_priorita"
Commit: "ded2bc7a7b5951137585515d917c48b4a15be6a1"
Test feedback: "Esercizio produttore-consumatore con priorita, con processi. L'esercizio non è ancora stato svolto correttamente. L'esecuzione del programma è andata in timeout"
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: procedure.c, function produci_bassa_prio (lines 75-115), the condition in the while loop (line 93) incorrectly increments and decrements p->num_produttori_alta_prio, which is intended to track high-priority producers. This violates the synchronization policy, as low-priority producers should not modify this variable. This leads to incorrect behavior, potentially causing a deadlock or incorrect prioritization, which explains the observed timeout. The correct approach would be to use a separate condition variable or mechanism to handle low-priority producer suspension without interfering with the high-priority producer count."

Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a concrete defect in the student's code, specifically the incorrect manipulation of p->num_produttori_alta_prio within the produci_bassa_prio function. This is supported by the diff, which shows the student code incrementing and decrementing p->num_produttori_alta_prio inappropriately in the while loop, whereas the solution does not. The diagnosis also aligns with the observed timeout failure, as the incorrect synchronization could lead to deadlock or improper prioritization. The diagnosis does not introduce unrelated or speculative issues and is consistent with the diff and feedback."

Esempio timeout 3

Studente: "chiaramen"
Esercizio: "produttore-consumatore_con_priorita"
Commit: "3257fe20fc9fc9121c50390f6efa2539bef0c7e8"
Test feedback: "Esercizio produttore-consumatore con priorita, con processi. L'esercizio non è ancora stato svolto correttamente. L'esecuzione del programma è andata in timeout"
LLM Code Correct: "NO"
LLM Code Diagnosis: "The function consuma in FILE: procedure.c (lines 111-149) does not implement the required signaling logic to prioritize high-priority producers when a buffer slot becomes available. Specifically, the exercise mandates that a high-priority producer waiting for a buffer must be signaled first, but the code lacks the necessary logic to check and prioritize high-priority producers (e.g., using queue_condition or ProdAltaPr). This omission can lead to a deadlock or starvation, as low-priority producers may block high-priority producers, causing the observed timeout during execution."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the missing signaling logic to prioritize high-priority producers when a buffer slot becomes available. This omission is evident in the diff, where the student code lacks the num_produttori_alta_prio tracking and conditional signaling lo-

gic (signal_condition(&p->m, NOT_FULL_1) for high-priority producers). The observed timeout failure is consistent with the described defect, as the absence of prioritization can lead to deadlock or starvation. The diagnosis aligns with the visible differences and the observed failure."

2.3 Esempi di risultati LLM nell'analisi di IPC leak

Esempio IPC leak 1

Studente: "MattCaccavale"
Esercizio: "lettori-scrittori_con_monitor_e_processi"
Commit: "8cc94a8c7ac65c409591fc31c3dd81ac711fb62f"
Test feedback: "Esercizio lettori-scrittori. L'esercizio non è ancora stato svolto correttamente. È necessario deallocare le risorse IPC al termine della esecuzione"
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: procedure.c, function FineLettura (lines 22-32), the condition if(p->num_lett == 0) is incorrect. The signal to condition variable 1 is triggered before decrementing p->num_lett. This can cause premature signaling to writers, violating the reader-writer synchronization policy. The decrement of p->num_lett should occur before checking and signaling to ensure proper coordination between readers and writers."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis claims an issue with the condition if(p->num_lett == 0) and the order of decrementing p->num_lett before signaling. However, the diff shows that the student code decrements p->num_lett after the signal, which matches the described defect. This is consistent with the observed failure and the synchronization policy violation described. The diagnosis aligns with the visible diff and feedback."

Esempio IPC leak 2

Studente: "alecsgal"
Esercizio: "prelievi_multipli"
Commit: "9ef45a96947a48fe878281362b283676c729555e"
Test feedback: "Esercizio prelievi multipli. L'esercizio non è ancora stato svolto correttamente. È necessario deallocare le risorse IPC al termine della esecuzione"
LLM Code Correct: "NO"
LLM Code Diagnosis: "The observed runtime failure (ipc_leak) is caused by the missing shmctl(shm_id, IPC_RMID, NULL) call in main.c after detaching the shared memory segment. While the shared memory is allocated and attached in lines 12 and 21, it is never explicitly marked for removal after use. This omission violates the cleanup/termination requirement for shared memory resources, leading to a resource leak. The correct cleanup step should be added after the rimuovi(p) call in line 95 to ensure proper deallocation of the shared memory."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the missing shmctl(shm_id, IPC_RMID, NULL) call for shared memory cleanup, which is explicitly visible in the diff. The observed feedback about IPC resource deallocation aligns with this omission. The diagnosis is specific, consistent with the diff, and matches the observed failure category (ipc_leak)."

2.4 Esempi di risultati LLM nell'analisi di dynamic failure

Esempio dynamic failure 1

Studente: "mickdicostanzo"
Esercizio: "prelievi_multipli"
Commit: "89a6705c25dacb3f07e73cb6918900eb091d279c"
Test feedback: "Esercizio prelievi multipli. L'esercizio non è ancora stato svolto correttamente. L'esecuzione non è corretta: La prima consumazione avviene senza che siano disponibili almeno due elementi nel buffer"

LLM Output Correct: "NO"
LLM Output Diagnosis: "The output is missing the second consumption value (val_2) for all consumer processes. Each consumer is expected to consume two elements per operation, but only the first value (val_1) is shown. This violates the protocol requiring consumers to consume two elements at a time."
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: prodcons.c, function consuma (lines 14-51), the second consumption value (val_2) is printed after signaling the producer condition (line 34). This premature signal allows producers to overwrite the buffer before val_2 is consumed, leading to missing or incorrect second consumption values in the output. The signal for producers should occur only after both val_1 and val_2 are consumed to maintain the FIFO protocol and ensure correct synchronization."
Judge Output Correct: "NO"
Judge Output Motivation: "The diagnosis identifies a missing second consumption value (val_2) for all consumer processes, which implies a protocol violation. However, the ground truth specifies that the issue is the first consumption occurring without at least two elements in the buffer. These are distinct failures: one concerns missing output values, while the other concerns a premature consumption operation. The diagnosis does not semantically align with the ground truth."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a synchronization issue in the consuma function of prodcons.c, specifically regarding the premature signaling of the producer condition after consuming val_1 but before consuming val_2. The diff shows that the student signals the producer condition (signal_condition(&p->MyMonitor,CV_PROD)) immediately after consuming val_1 (line 34), whereas the solution delays signaling until after both val_1 and val_2 are consumed. This aligns with the described defect mechanism and the observed test feedback about incorrect synchronization leading to protocol violations."

Esempio dynamic failure 2

Studente: “Fabronsky”
Esercizio: “produttore-consumatore_con_priorita”
Commit: “81f43129d3851726b87c324531e2fdac28941573”
Test feedback: “Esercizio produttore-consumatore con priorita, con processi. L’esercizio non è ancora stato svolto correttamente. L’esecuzione non è corretta: Il valore consumato in posizione 0 non coincide con il valore prodotto nella stessa posizione”
LLM Output Correct: “NO”
LLM Output Diagnosis: “The output shows mismatched counts and missing interactions. The high-priority producer is expected to produce 3 items but only produces 2. The consumer is expected to consume 12 items but consumes 11. Additionally, there is no concrete evidence that the low-priority producers respect the priority constraint, as they produce while the high-priority producer is still active. This violates the protocol requiring low-priority producers to wait if a high-priority producer is waiting for a buffer.”
LLM Code Correct: “NO”
LLM Code Diagnosis: “The function produci_bassa_prio in FILE: procedure.c (lines 72-109) contains a defect in the synchronization policy. The condition in line 89 (while(p->count == 0 || p->num_lettori_alta_priorita > 0)) is incorrect. It should check if the buffer is full (p->count == DIM) instead of empty (p->count == 0). This causes low-priority producers to produce even when the buffer is full or when high-priority producers are waiting, violating the priority constraint and leading to the observed mismatched counts and protocol violation in the runtime output.”
Judge Output Correct: “NO”
Judge Output Motivation: “The ground truth specifies an incorrect execution due to a mismatch between the produced and consumed values at position 0. The diagnosis identifies mismatched counts, missing interactions, and protocol violations, but it does not explicitly address the specific mismatch at position 0. Therefore, it does not semantically align with the ground truth’s failure description.”
Judge Code Correct: “YES”

Judge Code Motivation: "The diagnosis correctly identifies a defect in the synchronization policy of the produci_bassa_prio function, specifically the incorrect condition in the while loop at line 89. The diff shows that the student code checks for an empty buffer (p->count == 0) instead of a full buffer (p->count == DIM), which matches the described defect. The diagnosis also aligns with the observed runtime output indicating mismatched counts and protocol violations. The cited mechanism is consistent with the visible differences in the diff and the observed failure."

Esempio dynamic failure 3

Studente: "leonardomassa"
Esercizio: "produttore-consumatore_con_priorita"
Commit: "84e4f73ca4a470f50a9ca479ca432a4144c9ff27"
Test feedback: "Esercizio produttore-consumatore con priorita, con processi. L'esercizio non è ancora stato svolto correttamente. L'esecuzione non è corretta: Un produttore di tipo 2 produce mentre ci sono ancora produttori di tipo 1 in attesa. Il numero di valori prodotti non coincide con il numero di valori consumati"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output shows that the consumer consumes 12 items, but the total number of produced items is only 11 (3 from the high-priority producer and 8 from the low-priority producers). This mismatch in production and consumption counts indicates a protocol violation. Additionally, the high-priority producer's constraint of blocking low-priority producers when waiting for a buffer is not verifiable due to interleaving, but the count mismatch is a concrete issue."
LLM Code Correct: "NO"
LLM Code Diagnosis: "The function consuma in FILE: procedure.c (lines 115-161) fails to ensure that the total number of consumed items does not exceed the total number of produced items. Specifically, the consumer loop (lines 125-128) only checks if the buffer is empty before consuming, but it does not account for the total production count. This leads to a protocol violation, as evidenced by the runtime output where the consumer consumes 12 items while only 11 were produced. This mismatch indicates a failure to properly synchronize production and con-

sumption counts."
Judge Output Correct: "YES"
Judge Output Motivation: "The diagnosis correctly identifies the mismatch between the number of produced and consumed items, which aligns with the ground truth's statement about the production-consumption count discrepancy. Additionally, the diagnosis mentions the protocol violation involving producer priorities, which is consistent with the ground truth's note about type 2 producers producing while type 1 producers are waiting. Both issues are semantically aligned with the ground truth failures."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a concrete defect in the consumer loop in procedure.c (lines 125-128), where the student code fails to properly synchronize production and consumption counts. This is consistent with the observed feedback and the visible diff, which shows the absence of a wait_condition for the NOT_EMPTY signal in the student code. The diagnosis does not introduce unrelated or speculative issues and aligns with the structural difference in the diff."

2.5 Esempi di risultati LLM nell'analisi di static failure

Esempio static failure 1

Studente: "giuseppe-maglione"
Esercizio: "produttore-consumatore_con_priorita"
Commit: "0dc0fb4134033fd45227a4ec08bb8a7166367180"
Test feedback: "Esercizio produttore-consumatore con priorita, con processi. L'esercizio non è ancora stato svolto correttamente. Anche se il programma esegue, sono stati riscontrati i seguenti difetti all'interno del codice (procedure.c)"
Static warnings: "["file": "procedure.c", "line": 95, "message": "È un monitor signal and continue, è necessario utilizzare wait_condition() all'interno di un ciclo."]"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output shows mismatched counts and missing interactions. The high-priority producer (pid

349782) produces only 3 values, but the consumer consumes 12 values, leaving 9 unaccounted for. Additionally, the low-priority producers produce values even when the high-priority producer is active, violating the priority constraint. The trace also lacks clear concrete evidence of proper buffer management or adherence to the described protocol."
LLM Code Correct: "NO"
LLM Code Diagnosis: "FILE: procedure.c, function produci_bassa_prio, lines 83-85: The synchronization policy for low-priority producers violates the priority constraint. Specifically, the condition p->count == DIM || p->num_alta_prio > 0 only suspends low-priority producers if high-priority producers are already waiting. However, it does not ensure that high-priority producers are always prioritized when both types are ready to produce. This violates the required producer-consumer priority policy."
Judge Output Correct: "NO"
Judge Output Motivation: "The ground truth explicitly states that the visible execution/output is correct, while the diagnosis claims multiple mismatches, missing interactions, and protocol violations. These claims contradict the ground truth and invent problems that do not exist in the output."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a synchronization policy issue in produci_bassa_prio that violates the priority constraint, which is causally related to the ground truth warnings about improper use of wait_condition() in a loop. The diagnosis provides a detailed explanation of the defect mechanism, staying within the same defect family (synchronization and priority handling)."

Esempio static failure 2

Studente: "luke-the-coder"
Esercizio: "produttore-consumatore_con_priorita"
Commit: "d74b3dea75ebee4f713ce6cd928f9b77f93e53d2"
Test feedback: "Esercizio produttore-consumatore con priorita, con processi. L'esercizio non è ancora stato svolto correttamente.

Anche se il programma esegue, sono stati riscontrati i seguenti difetti all'interno del codice (procedure.c)"
Static warnings: "["file": "procedure.c", "line": 94, "message": "È un monitor signal and continue, è necessario utilizzare wait_condition() all'interno di un ciclo."]"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output shows inconsistencies in the protocol. Specifically, the high-priority producer (pid 352323) is shown to have completed its three productions, but low-priority producers (pid 352324, 352325, 352326) continue producing while there are still processes in the high-priority queue (e.g., "Processi in coda nella variabile PROD_ALTA_PRIORITA 1"). This violates the requirement that low-priority producers can only produce when no high-priority producers are waiting for a buffer. Additionally, the total number of consumed items (12) matches the expected count, but the pairing of produced and consumed values is not fully verifiable."
LLM Code Correct: "NO"
LLM Code Diagnosis: "The synchronization policy for the produci_bassa_prio function in FILE: procedure.c (lines 68-105) is incorrect. While the function correctly checks if the buffer is full (p->count == DIM) or if a high-priority producer is waiting (p->num_lettori_alta_priorita > 0), it does not signal a high-priority producer when leaving the monitor if the buffer becomes available. This violates the priority producer-consumer constraint that low-priority producers must yield to high-priority producers waiting for a buffer. The defect lies in the absence of a conditional signal to PROD_ALTA_PRIORITA in this function."
Judge Output Correct: "NO"
Judge Output Motivation: "The ground truth explicitly states that the visible execution/output is correct, while the diagnosis claims specific protocol violations and inconsistencies in the output. This directly contradicts the ground truth, which confirms correctness. The diagnosis invents problems not supported by the ground truth."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis identifies a synchronization issue related to the use of conditional signaling in the pro-

duci_bassa_prio function, which aligns with the ground truth warnings about improper use of "signal and continue" and the need for wait_condition() in a loop. Both the diagnosis and the ground truth warnings pertain to the same defect mechanism involving synchronization and signaling in the monitor."

Esempio static failure 3

Studente: "Marcodlc01-1"
Esercizio: "prelievi_multipli"
Commit: "2eafe209dc772ce8119ac0bd3946e4601c34c55d"
Test feedback: "Esercizio prelievi multipli. L'esercizio non è ancora stato svolto correttamente. Anche se il programma esegue, sono stati riscontrati i seguenti difetti all'interno del codice (prodcons.c)"
Static warnings: "["file": "prodcons.c", "line": 78, "message": "La funzione signal_condition va chiamata due volte nel consumatore, perché sono fatte due consumazioni."]"
LLM Output Correct: "YES"
LLM Output Diagnosis: "No concrete output mismatch is visible. The trace structure is consistent with the exercise protocol. All producers and consumers are active, and their interactions follow the expected FIFO buffer behavior. Producers produce one value at a time, and consumers consume two values per operation. The counts, pairing, and ordering of operations are coherent, and no missing or malformed interactions are observed."
LLM Code Correct: "NO"
LLM Code Diagnosis: "FILE: prodcons.c, function produci, lines 64-68. The synchronization policy for the producer is incorrect. The producer uses wait_condition(&(p->m), MESS_DISP) when the buffer is full (line 64), but this condition variable is intended for consumers waiting for messages to be available. The correct condition variable for producers to wait on should be SPAZIO_DISP, as indicated by the comment on line 69. This violates the specified synchronization policy for the monitor and may lead to incorrect behavior."
Judge Output Correct: "YES"
Judge Output Motivation: "The diagnosis correctly states

that no concrete output mismatch is visible and aligns with the ground truth, which confirms the output is correct. It does not introduce any invented problems or contradictions."
Judge Code Correct: "NO"
Judge Code Motivation: "The diagnosis identifies an incorrect synchronization policy for the producer, specifically the use of the wrong condition variable (MESS_DISP instead of SPAZIO_DISP) when the buffer is full. However, the ground truth warning pertains to the consumer's signaling behavior and does not mention or imply any defect in the producer's synchronization policy. These are distinct defect families, and the diagnosis does not address the ground truth warning."

2.6 Esempi di risultati LLM nell'analisi di correct

Esempio correct 1

Studente: "Darklord1470"
Esercizio: "lettori-scrittori_con_monitor_e_processi"
Commit: "39d98f71a25b76dd2b37a9656fd24d5435170a13"
Test feedback: "Esercizio lettori-scrittori.Congratulazioni, l'esercizio compila ed esegue correttamente, e non è stato riscontrato nessuno degli errori frequenti nel codice"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output shows that all initial readings by user processes report the same values (Temperature=0, Humidity=0, Rain=no) repeatedly, which is inconsistent with the exercise requirement that the meteorological data should be updated randomly by process M every 2 seconds. Additionally, the writing operations by process M appear only at the end of the output, after all user readings, which violates the expected protocol of concurrent updates and reads. This indicates a mismatch in the timing and coherence of the producer/consumer protocol."
LLM Code Correct: "NO"
LLM Code Diagnosis: "The function InizioLettura in procedure.c contains a logic error. Specifically, the condition if(p->sovrascrittura > 0) followed by wait_condition(&(p->m), LETTORI) is incorrect because it may block readers unnecessarily

when no writers are active. This violates the reader-writer protocol, where multiple readers should be allowed simultaneous access unless a writer is active."
Judge Output Correct: "NO"
Judge Output Motivation: "The ground truth explicitly states that the visible execution/output is correct, while the diagnosis claims specific timing and coherence issues in the output, which contradicts the ground truth. The diagnosis invents problems not supported by the ground truth."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a logic error in the InizioLettura function. The student code uses if(p->sovrascrittura > 0) followed by wait_condition(&(p->m), LETTORI), which is semantically different from the solution's while (p->num_scrittori>0 || queue_condition(&p->monitor_meteo,CV_METEO)>0). This difference supports the claim that the student code may block readers unnecessarily, violating the reader-writer protocol. The feedback does not contradict this, and the diff provides concrete evidence for the claimed defect mechanism."

Esempio correct 2

Studente: "FrancescoDon-1"
Esercizio: "lettori-scrittori_con_monitor_e_processi"
Commit: "dbeb51fd37d2c2957591464da108313f477923ad"
Test feedback: "Esercizio lettori-scrittori.Congratulazioni, l'esercizio compila ed esegue correttamente, e non è stato riscontrato nessuno degli errori frequenti nel codice"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output shows mismatched counts and protocol violations. The writer process (PID 3574567) performs 20 writes as expected, but the reader processes do not perform the required 10 reads each. Some readers perform fewer than 10 reads, and there are inconsistencies in the number of reads across different reader processes. Additionally, there are duplicate readings of the same data by the same process, which suggests improper synchronization or protocol handling. The trace structure is incomplete and does not fully adhere to the exercise protocol."

LLM Code Correct: "NO"
LLM Code Diagnosis: "The function InizioScrittura in procedure.c does not properly block when num_scrittori > 0, as it only checks num_lettori > 0 before calling wait_condition. This violates the mutual exclusion requirement for the writer process (M), potentially allowing multiple writers to access the shared resource simultaneously."
Judge Output Correct: "NO"
Judge Output Motivation: "The ground truth explicitly states that the visible execution/output is correct, while the diagnosis claims multiple mismatches, protocol violations, and synchronization issues. These claims contradict the ground truth and invent problems that are not present."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis claims that the function InizioScrittura does not properly block when num_scrittori > 0, which could violate mutual exclusion for writers. The diff shows that the student's implementation only checks num_lettori > 0 before calling wait_condition, while the solution checks both num_utenti > 0 and num_scrittori > 0 in a while loop. This difference directly supports the diagnosis, as the student's code does not account for the num_scrittori > 0 condition, potentially allowing multiple writers to access the shared resource simultaneously. The observed feedback does not contradict this finding, as it does not explicitly confirm correctness in this specific aspect."

Esempio correct 3

Studente: "Furbizio"
Esercizio: "prelievi_multipli"
Commit: "997e85f3361478741f025b6108b12a8b9d231eb0"
Test feedback: "Esercizio prelievi multipli.Congratulazioni, l'esercizio compila ed esegue correttamente, e non è stato riscontrato nessuno degli errori frequenti nel codice"
LLM Output Correct: "YES"
LLM Output Diagnosis: "No concrete output mismatch is visible. The trace structure is consistent with the exercise protocol. Producers produce one element at a time, and consumers consume

two elements per operation. The counts, pairings, and ordering of production and consumption events are coherent, and no missing interactions or invalid values are observed. All phases, including process initialization, production, consumption, and termination, are present in the output."
LLM Code Correct: "YES"
LLM Code Diagnosis: "No concrete correctness concrete defect is visible from the provided code and runtime context. The program adheres to the explicit requirements of the exercise, including the use of a Hoare monitor for synchronization, proper producer and consumer behavior, and correct handling of shared memory. The observed output confirms that all producers and consumers execute their tasks as expected, with no structural issues or protocol violations evident."
Judge Output Correct: "YES"
Judge Output Motivation: "The diagnosis correctly states that no concrete output mismatch is visible and aligns with the ground truth, which confirms the output is correct. No invented problems are added, and the detailed description enriches the generic correctness claim without contradiction."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly states that no concrete correctness defect is visible in the provided code and runtime context. The observed feedback confirms the program compiles and executes correctly without errors, and the diff does not show any explicit correctness violations. The differences in the diff are superficial and do not constitute a correctness defect. The diagnosis aligns with the rules and observed feedback."

3 Esempi analisi LLM esercitazione 3

3.1 Esempi di risultati LLM nell'analisi di crash

Esempio crash 1

Studente: "Z3R0M3M"
Esercizio: "lettori-scrittori_su_piu_oggetti_monitor"
Commit: "d6619d601447e0ee001780bfdb36ff84f5c64a57"

Test feedback: “Esercizio lettori-scrittori su piu oggetti monitor. L’esercizio non è ancora stato svolto correttamente. L’esecuzione del programma è andata in crash”
LLM Code Correct: “NO”
LLM Code Diagnosis: “In FILE: main.c, lines 60-89, the program fails to allocate and initialize the 4 monitor instances (struct monitor* m[4]) before accessing them (e.g., m[0]->id_treno=1 on line 66). This leads to undefined behavior when dereferencing uninitialized pointers, causing the observed crash. The missing allocation and initialization steps violate the ownership/lifetime requirement for the monitor objects.”
Judge Code Correct: “YES”
Judge Code Motivation: “The diagnosis correctly identifies the missing allocation and initialization of the struct monitor* m[4] array before dereferencing its elements (e.g., m[0]->id_treno=1 on line 66). This defect is explicitly visible in the diff, where the solution includes a loop to allocate memory for m[i] (lines 64-66), which is absent in the student’s code. The observed crash is consistent with undefined behavior caused by dereferencing uninitialized pointers. The diagnosis matches the concrete mechanism visible in the diff and aligns with the observed failure.”

Esempio crash 2

Studente: “Vecchions110”
Esercizio: “lettori-scrittori_su_piu_oggetti_monitor”
Commit: “9da2cacc8970ee7b0544ee0bd5afcef7972f7e8c”
Test feedback: “Esercizio lettori-scrittori su piu oggetti monitor. L’esercizio non è ancora stato svolto correttamente. L’esecuzione del programma è andata in crash”
LLM Code Correct: “NO”
LLM Code Diagnosis: “In FILE: main.c, lines 60-69, the array m is used to assign id_treno values to the monitor instances (lines 66-69) before the monitors are allocated and initialized. This results in undefined behavior, as m[0], m[1], m[2], and m[3] are uninitialized pointers. This likely causes the crash observed during runtime. The monitors must be properly allocated (e.g.,

using malloc) and initialized with inizializza() before accessing or modifying their fields."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the concrete issue of undefined behavior due to uninitialized pointers in the array m being dereferenced before allocation and initialization. This is directly supported by the diff, which shows the absence of memory allocation (malloc) for m[i] in the student's code, leading to the crash observed during runtime. The diagnosis does not introduce unrelated or speculative issues and aligns with the visible differences in the diff and the observed failure."

Esempio crash 3

Studente: "leonardoricc2002"
Esercizio: "lettori-scrittori_su_piu_oggetti_monitor"
Commit: "05cb524a0aff6027092f946097368d8c7ca7c5c0"
Test feedback: "Esercizio lettori-scrittori su piu oggetti monitor. L'esercizio non è ancora stato svolto correttamente. L'esecuzione del programma è andata in crash"
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: main.c, lines 60-69, the m array of struct monitor pointers is declared but not initialized with allocated memory. The code directly assigns values to m[i]->id_treno (lines 66-69) without allocating memory for m[i] or calling inizializza(). This leads to undefined behavior when dereferencing uninitialized pointers, causing the observed crash. The defect violates the ownership/lifetime invariant, as the required allocation and initialization of the monitor instances are missing."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the missing memory allocation for the m array elements and the subsequent undefined behavior when dereferencing uninitialized pointers. This matches the concrete structural difference visible in the diff (missing malloc loop in the student code) and is consistent with the observed crash feedback. The diagnosis does not introduce unrelated or speculative issues and aligns with the rules for crash failures."

3.2 Esempi di risultati LLM nell'analisi di timeout

Esempio timeout 1

Studente: "Mettaton12"
Esercizio: "lettori-scrittori_su_piu_oggetti_monitor"
Commit: "dc0680fc875a9e7eb8b90294c52a9b0d3bafdeb3"
Test feedback: "Esercizio lettori-scrittori su piu oggetti monitor. L'esercizio non è ancora stato svolto correttamente. L'esecuzione del programma è andata in timeout"
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: procedure.c, function leggi_stazione, line 96 contains a critical defect: if(m->n_lettori=0) uses an assignment operator = instead of a comparison operator ==. This causes incorrect behavior in the reader-writer synchronization logic, as the condition always evaluates to true, leading to premature signaling of writers and potential deadlock or timeout. This directly explains the observed runtime timeout, as the synchronization policy for readers and writers is violated."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a critical defect in procedure.c, function leggi_stazione, where the assignment operator = is used instead of the comparison operator == in the condition if(m->n_lettori=0). This specific line is visible in the diff and matches the described defect mechanism, which directly explains the observed timeout due to synchronization issues. The cited file, function, and line are compatible with the diff, and the defect is clearly supported by the evidence."

Esempio timeout 2

Studente: "gabrieleesposito11"
Esercizio: "lettori-scrittori_su_piu_oggetti_monitor"
Commit: "d9177e0d2fd908a8ae51845d4f8d23efdb0c546e"
Test feedback: "Esercizio lettori-scrittori su piu oggetti monitor. L'esercizio non è ancora stato svolto correttamente. L'esecuzione del programma è andata in timeout"
LLM Code Correct: "NO"

LLM Code Diagnosis: "In FILE: main.c, lines 103 and 109, the pthread_join calls for the "Capotreno" and "Viaggiatori" threads are incorrect. The pthread_join function expects a thread ID as its first argument, but the code incorrectly passes the address of the thread ID (&capo[i] and &viagg[i]) instead of the thread ID itself (capo[i] and viagg[i]). This causes the program to fail to properly wait for the threads to complete, leading to a potential timeout as observed in the runtime output. This violates the thread synchronization policy."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the issue with the pthread_join calls in the student's code. The diff explicitly shows that the student incorrectly passes the address of the thread ID (&capo[i] and &viagg[i]) instead of the thread ID itself (capo[i] and viagg[i]), which matches the described defect. This is a concrete structural difference visible in the diff and consistent with the observed timeout failure due to improper thread synchronization. The diagnosis does not introduce unrelated or speculative issues and aligns with the rules."

Esempio timeout 3

Studente: "GiuseppeFus"
Esercizio: "lettori-scrittori_su_piu_oggetti_monitor"
Commit: "001affe1a6193f468dd394af7cec2e4502d0960b"
Test feedback: "Esercizio lettori-scrittori su piu oggetti monitor. L'esercizio non è ancora stato svolto correttamente. L'esecuzione del programma è andata in timeout"
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: procedure.c, function scrivi_stazione, the pthread_mutex_unlock call at line 53 is misplaced. The m->occupato flag is set to 1 at line 52, but the mutex is unlocked immediately after, leaving the critical section unprotected during the write operation at line 57. This violates the reader-writer synchronization policy, as concurrent readers or writers could access shared state unsafely, potentially causing a deadlock or race condition. This defect likely explains the observed timeout during execution. The mutex should remain locked

until the critical section is fully completed."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a misplaced pthread_mutex_unlock call in the student's code, which prematurely unlocks the mutex after setting m->occupato=1 at line 52. This leaves the critical section unprotected during the write operation at line 57, as shown in the diff. The diagnosis aligns with the observed timeout failure and is consistent with the visible structural difference in synchronization logic between the student and solution code. The defect mechanism is concrete and matches the diff."

3.3 Esempi di risultati LLM nell'analisi di dynamic failure

Esempio dynamic failure 1

Studente: "MarcoDani"
Esercizio: "produttore-consumatore_con_priorita"
Commit: "1e754101b286de9f6279ae2e1362a86208a164b5"
Test feedback: "Esercizio produttore-consumatore con priorita, con thread. L'esercizio non è ancora stato svolto correttamente. L'esecuzione non è corretta: Il valore consumato in posizione 2 non coincide con il valore prodotto nella stessa posizione"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output shows mismatched counts and ordering issues. The high-priority producer (Produttore tipo 1) produces only 3 items, but the consumer consumes 12 items, with some values being repeated (e.g., 0 and 5). This suggests either missing production events or incorrect buffer management. Additionally, the protocol requiring low-priority producers to wait for high-priority producers is not clearly enforced, as low-priority producers (Produttore tipo 2) produce while high-priority producers are active. The trace does not fully adhere to the exercise protocol."
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: prodcons.c, function consuma (lines 110-137), the buffer index used for consumption (p-

>coda) is incorrectly updated before retrieving the value, causing repeated consumption of the same value. Specifically, value = p->buffer[p->coda] (line 126) should occur before p->coda = (p->coda + 1) % DIM (line 127). This violates the circular buffer protocol, leading to incorrect output where values are consumed multiple times, as seen in the runtime trace (e.g., repeated consumption of 0 and 5)."
Judge Output Correct: "NO"
Judge Output Motivation: "The diagnosis identifies mismatched counts, ordering issues, and protocol violations, which are not explicitly mentioned in the ground truth. The ground truth specifies a specific mismatch between produced and consumed values at a particular position, which is a different category of failure. The diagnosis does not semantically align with the ground truth's focus on value mismatches at specific positions."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a concrete defect in the consuma function where the buffer index (p->coda) is updated before retrieving the value, leading to repeated consumption of the same value. This matches the observed failure and is consistent with the visible diff. The diagnosis does not introduce unrelated or speculative issues and aligns with the rules."

Esempio dynamic failure 2

Studente: "angy0923-1"
Esercizio: "una_struttura_dati_stack_thread-safe"
Commit: "6eccce3c53496d4fe807e91bbe8c69a1e18f016c"
Test feedback: "Esercizio struttura dati thread-safe. L'esercizio non è ancora stato svolto correttamente. L'esecuzione non è corretta: Il valore prelevato non rispetta la politica LIFO dello stack. Al termine dell esecuzione lo stack non risulta vuoto"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output contains several inconsistencies. For example, "Dimensione" values are incorrect or nonsensical at times (e.g., 32616, 18, 15) and do not align with the expected stack size changes based on the number of "Inseri-

mento" and "Prelievo" operations. Additionally, some "Prelievo" operations appear to retrieve the same value multiple times (e.g., "Prelievo: 4" twice consecutively), which violates stack behavior. These issues indicate protocol violations in the stack operations."
LLM Code Correct: "NO"
LLM Code Diagnosis: "The function StackSize in FILE: stack.c (lines 79-90) does not synchronize correctly with other stack operations. While it uses a mutex to protect access to s->testa, it does not account for concurrent modifications by StackPush or StackPop. This can lead to inconsistent or stale values being read, as seen in the output where "Dimensione" values are nonsensical (e.g., 32616, 18, 15). The defect violates the thread-safe requirement of the stack, causing protocol mismatches and incorrect runtime behavior."
Judge Output Correct: "YES"
Judge Output Motivation: "The diagnosis aligns with the ground truth by identifying violations of stack behavior, specifically mentioning LIFO policy issues and mismatched stack state, which are consistent with the ground truth's description of incorrect execution and a non-empty stack at the end. The detailed examples enrich the generic ground truth without contradicting it."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a synchronization issue in the StackSize function, which is consistent with the observed failure (nonsensical "Dimensione" values) and the diff. The diff shows that the student code uses different mutex and condition variable names (s->m, s->ok_push, s->ok_pop) compared to the solution (s->mutex, s->cv_produttore, s->cv_consumatore), but this is a superficial difference. The diagnosis focuses on the lack of proper synchronization in StackSize, which is not directly visible in the provided diff but aligns with the observed failure and the thread-safety requirement. The diagnosis does not introduce unrelated or speculative defects."

Esempio dynamic failure 3

Studente: “ManuelParasmo”
Esercizio: “produttore-consumatore_con_priorita”
Commit: “0b395ef14d0239c9a89d14c21cc1f19a78efe154”
Test feedback: “Esercizio produttore-consumatore con priorita, con thread. L’esercizio non è ancora stato svolto correttamente. L’esecuzione non è corretta: Il numero di valori prodotti non coincide con il numero di valori consumati”
LLM Output Correct: “NO”
LLM Output Diagnosis: “The output shows only "Produttore tipo 2" (low-priority producers) and the consumer, with no concrete evidence of "Produttore tipo 1" (high-priority producer) activity. This violates the protocol requiring both producer types to participate. Additionally, the high-priority producer’s precedence over low-priority producers is not demonstrated. The trace is incomplete and inconsistent with the exercise requirements.”
LLM Code Correct: “NO”
LLM Code Diagnosis: “In FILE: main.c, the thread creation logic at lines 95-100 incorrectly creates four low-priority producer threads (produttore_bassa_priorita) instead of three, and no high-priority producer thread (produttore_alta_priorita) is created. This violates the exercise requirement to have one high-priority producer and three low-priority producers. Consequently, the observed output lacks any activity from "Produttore tipo 1", leading to a protocol mismatch where high-priority producers do not participate, and their precedence over low-priority producers is not enforced.”
Judge Output Correct: “NO”
Judge Output Motivation: “The diagnosis identifies a different failure (missing high-priority producer activity and precedence issues) than the ground truth, which specifies a mismatch between the number of produced and consumed values. These are distinct categories of failure.”
Judge Code Correct: “YES”
Judge Code Motivation: “The diagnosis correctly identifies that the student code creates four low-priority producer threads instead of three and omits the creation of a high-priority producer thread. This is consistent with the visible diff and the observed

failure, which indicates a mismatch in the producer-consumer protocol. The diagnosis does not introduce unrelated or unsupported defects and aligns with the concrete structural difference in the diff."

3.4 Esempi di risultati LLM nell'analisi di static failure

Esempio static failure 1

Studente: "AMprota"
Esercizio: "produttore-consumatore_con_priorita"
Commit: "53454f309deac0afd091558aed3c6f7898a2a49d"
Test feedback: "Esercizio produttore-consumatore con priorita, con thread. L'esercizio non è ancora stato svolto correttamente. Anche se il programma esegue, sono stati riscontrati i seguenti difetti all'interno del codice (prodcons.c)"
Static warnings: "["file": "prodcons.c", "line": 78, "message": "La condizione va controllata all'interno di un ciclo "while". Nei programmi basati su libreria PThreads, il monitor è sempre di tipo signal-and-continue.","file": "prodcons.c", "line": 183, "message": "È necessario controllare che il consumatore verifichi la presenza di produttori ad alta priorità in attesa e risvegliare loro. Altrimenti, il consumatore risveglia un produttore a bassa priorità."]"
LLM Output Correct: "NO"
LLM Output Diagnosis: "FILE: prodcons.c, function produci_bassa_prio, lines 110-118: The synchronization policy for low-priority producers is incomplete. The pthread_cond_wait is correctly used to suspend low-priority producers when the buffer is full or high-priority producers are waiting. However, after resuming, the condition (p->size == DIM || p->threads_prio_1 > 0) is not rechecked before proceeding, which violates the monitor's signal-and-continue semantics and risks incorrect behavior if the condition is no longer valid."
Judge Output Correct: "NO"
Judge Output Motivation: "The diagnosis claims mismatched counts, protocol violations, and buffer management issues, which

directly contradict the ground truth that states the output is correct. The diagnosis invents problems not present in the ground truth."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the same defect mechanism as the ground truth warnings: the failure to recheck the condition after resuming from pthread_cond_wait, which violates the signal-and-continue semantics of PThreads monitors. It provides a detailed explanation of the issue within the same defect family and does not introduce unrelated problems. The line numbers and specific function mentioned in the diagnosis align with the logical code region and mechanism described in the warnings."

Esempio static failure 2

Studente: "P0l1702"
Esercizio: "lettori-scrittori_su_piu_oggetti_monitor"
Commit: "8674de407c198b58f95101f13d79b3901004741b"
Test feedback: "Esercizio lettori-scrittori su piu oggetti monitor. L'esercizio non è ancora stato svolto correttamente. Anche se il programma esegue, sono stati riscontrati i seguenti difetti all'interno del codice (procedure.c)"
Static warnings: "["file": "procedure.c", "line": 54, "message": "In questo esercizio si raccomanda di utilizzare l'algoritmo con starvation di entrambi (sia lettori sia scrittori). Per realizzare questo algoritmo, è necessario che la scrittura sia effettuata uscendo dal monitor (ossia, liberando temporaneamente la sezione critica del monitor, e riacquisendola subito dopo la scrittura). Questo consente ad eventuali altri scrittori di porsi in attesa su una variabile condition, e di potergli dare la priorità quando lo scrittore deve risvegliare un altro thread.", "file": "procedure.c", "line": 92, "message": "È necessario che la lettura sia effettuata uscendo dal monitor (ossia, liberando temporaneamente la sezione critica del monitor, e riacquisendola subito dopo la lettura). Questo consente a più lettori di poter leggere contemporaneamente il buffer."]"
LLM Output Correct: "YES"

LLM Output Diagnosis: "No concrete output mismatch is visible. The trace structure is consistent with the exercise protocol. All expected actors (capitreno and viaggiatori) are present, and their interactions (scrittura and lettura) follow the described protocol. The values for train positions increment correctly during scrittura, and lettura operations reflect the current positions without any visible inconsistencies. The counts, pairing, and ordering of operations are coherent with the exercise requirements."
Judge Output Correct: "YES"
Judge Output Motivation: "The diagnosis correctly states that no concrete output mismatch is visible and aligns with the ground truth, which confirms the output is correct. It does not invent any additional problems or contradict the ground truth."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the defect mechanism described in the ground truth warning on line 92. Both the diagnosis and the warning address the issue of improper reader synchronization, specifically the failure to allow multiple readers to read concurrently due to incorrect handling of the monitor's critical section. The diagnosis provides additional detail about the root cause (m->occupato flag), which is consistent with the ground truth's requirement for concurrent reader access. The diagnosis does not contradict or invent a different defect family."

Esempio static failure 3

Studente: "alecsgal"
Esercizio: "produttore-consumatore_con_priorita"
Commit: "d12401b1abe97cea1664127c2af3d96fde7a9f42"
Test feedback: "Esercizio produttore-consumatore con priorita, con thread. L'esercizio non è ancora stato svolto correttamente. Anche se il programma esegue, sono stati riscontrati i seguenti difetti all'interno del codice (prodcons.c)"
Static warnings: "["file": "prodcons.c", "line": 60, "message": "In questo esercizio, è necessario che i produttori incrementino una variabile di conteggio subito prima di chiamare cond_wait(), e che la decrementino subito dopo la chiamata a cond_wait().

Queste variabili di conteggio consentono di sapere se ci sono thread ad alta priorità in attesa, e di dargli la precedenza quando il consumatore deve risvegliare dei produttori.", "file": "prodcons.c", "line": 128, "message": "La condizione va controllata all'interno di un ciclo "while". Nei programmi basati su libreria PThreads, il monitor è sempre di tipo signal-and-continue."]"
LLM Output Correct: "NO"
LLM Output Diagnosis: "FILE: prodcons.c, function produci_alta_prio, line 48: The synchronization policy is violated because p->coda_alta_priorita is incremented after the thread is already waiting on pthread_cond_wait. This causes incorrect tracking of high-priority producers waiting for a buffer, as the increment should occur before entering the wait state to ensure accurate priority handling."
Judge Output Correct: "NO"
Judge Output Motivation: "The diagnosis claims mismatched counts, incomplete interactions, and protocol violations, which directly contradict the ground truth stating the output is correct. The diagnosis invents problems not present in the ground truth."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the same underlying defect mechanism as the ground truth warning on line 60, specifically the incorrect placement of the increment operation relative to pthread_cond_wait. Both focus on the synchronization policy violation and its impact on priority handling. The diagnosis does not contradict or invent a different defect family."

3.5 Esempi di risultati LLM nell'analisi di correct

Esempio correct 1

Studente: "Claudio-Cataldo"
Esercizio: "produttore-consumatore_con_priorita"
Commit: "396b08b449ef801a2db78929bced63bb5673aacb"
Test feedback: "Esercizio produttore-consumatore con priorita, con thread.Congratulazioni, l'esercizio compila ed esegue correttamente, e non è stato riscontrato nessuno degli errori frequenti

nel codice"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output shows inconsistencies in the producer-consumer protocol. Specifically, the buffer count ("NUMERO ELEMENTI") remains at 3 for multiple consecutive consume operations, which is logically impossible as each consume should decrement the buffer count. Additionally, the buffer count decreases only after several consume operations, indicating a mismatch in buffer management. This violates the expected protocol coherence and value consistency."
LLM Code Correct: "NO"
LLM Code Diagnosis: "The function produci_bassa_prio in prodcons.c does not enforce the requirement that low-priority producers must wait if there are high-priority producers waiting for a buffer. The condition while (p->num_elem == DIM) in line 93 only checks if the buffer is full but does not account for the presence of high-priority producers (p->num_alta_prio > 0). This violates the explicit exercise requirement that low-priority producers must defer to high-priority producers in such cases."
Judge Output Correct: "NO"
Judge Output Motivation: "The ground truth explicitly states that the visible execution/output is correct, while the diagnosis claims there are inconsistencies in the producer-consumer protocol and buffer management. This directly contradicts the ground truth, which confirms no issues in the output. Therefore, the diagnosis does not align with the ground truth."
Judge Code Correct: "NO"
Judge Code Motivation: "The diagnosis claims that the function produci_bassa_prio in prodcons.c does not enforce the requirement for low-priority producers to wait if high-priority producers are waiting. However, the diff does not include the implementation of produci_bassa_prio or any relevant synchronization logic in prodcons.c. Without visibility into the specific code for this function or its synchronization mechanism, the diagnosis is speculative. The observed test feedback also reports no errors, further undermining the claim."

Esempio correct 2

Studente: “chia-me”
Esercizio: “produttore-consumatore_con_priorita”
Commit: “25038956218dd7dfe26c998fa1df808a44fb88ee”
Test feedback: “Esercizio produttore-consumatore con priorita, con thread.Congratulazioni, l’esercizio compila ed esegue correttamente, e non è stato riscontrato nessuno degli errori frequenti nel codice”
LLM Output Correct: “NO”
LLM Output Diagnosis: “The output shows 12 consume operations, which matches the expected count, but there are inconsistencies in the producer-consumer protocol. Specifically, "Produttore tipo 2" produces more than 9 items (expected 3 items per 3 threads), and "Produttore tipo 1" produces fewer than 3 items. This violates the expected counts per producer type. Additionally, the priority constraint is not visibly enforced, as "Produttore tipo 2" produces while "Produttore tipo 1" is active or waiting. These issues indicate a mismatch in counts and priority handling.”
LLM Code Correct: “NO”
LLM Code Diagnosis: “In prodcons.c, the function produci_bassa_prio contains a concrete defect in the condition variable handling. Specifically, in the while loop at line 93, the pthread_prio_2 counter is incremented twice (lines 94 and 96) without decrementing, leading to an incorrect count of low-priority producers waiting. This can cause incorrect signaling behavior and violate the priority constraint.”
Judge Output Correct: “NO”
Judge Output Motivation: “The ground truth explicitly states that the visible execution/output is correct, while the diagnosis claims multiple issues with the output, including mismatched counts and priority handling. These claims contradict the ground truth, which confirms correctness. Therefore, the diagnosis does not align with the ground truth.”
Judge Code Correct: “YES”
Judge Code Motivation: “The diagnosis claims a defect in the condition variable handling in producibassa_prio, specifically that the pthread_prio_2 counter is incremented twice without

decrementing, leading to an incorrect count. The diff confirms this issue: the student code increments pthread_prio_2 twice (lines 94 and 96) but does not decrement it after the pthread_cond_wait call, unlike the solution which correctly increments before and decrements after the wait. This matches the described defect mechanism. The observed feedback does not report any issues, but the diff explicitly shows a correctness violation."

Esempio correct 3

Studente: "gabriele-brancale"
Esercizio: "lettori-scrittori_su_piu_oggetti_monitor"
Commit: "97c6a2ce48c757186962d89ed4f2f340ec9d0b69"
Test feedback: "Esercizio lettori-scrittori su piu oggetti monitor.Congratulazioni, l'esercizio compila ed esegue correttamente, e non è stato riscontrato nessuno degli errori frequenti nel codice"
LLM Output Correct: "YES"
LLM Output Diagnosis: "The output is consistent with the exercise protocol. All expected actors (capitreno and viaggiatori) are present, and their interactions follow the described request/-reply structure. The positions of the trains are updated correctly and sequentially, and the viaggiatori read the expected positions. All threads terminate as expected, and no concrete mismatches, missing phases, or protocol violations are visible."
LLM Code Correct: "NO"
LLM Code Diagnosis: "The function leggi_stazione in procedure.c contains a concrete defect where the m->occupato flag is set to 1 when a reader accesses the monitor (line 78). This violates the reader-writer problem requirement that multiple readers should be allowed to access the resource concurrently. Setting m->occupato to 1 prevents other readers from accessing the resource simultaneously, which is an explicit violation of the exercise requirement to allow concurrent reads."
Judge Output Correct: "YES"
Judge Output Motivation: "The diagnosis correctly states that the output is consistent with the exercise protocol and confirms no visible mismatches, which aligns with the ground truth stating the output is correct. No invented problems are added."

Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a concrete defect in the leggi_stazione function where m->occupato is set to 1, preventing concurrent readers, which violates the reader-writer problem requirements. This is directly supported by the diff, as the student code sets m->occupato = 1 (line 78), while the solution uses a different synchronization mechanism that allows multiple readers (m->num_lettori++ and no m->occupato flag). The feedback does not contradict this, and the cited mechanism aligns with the visible code difference."

4 Esempi analisi LLM esercitazione 4

4.1 Esempi di risultati LLM nell'analisi di timeout

Esempio timeout 1

Studente: "giovanni158"
Esercizio: "server_sincroni_multipli"
Commit: "d793a34970d61daeefee1b1835bde5cdc9e7cb1a"
Test feedback: "Esercizio server sincroni multipli. L'esercizio non è ancora stato svolto correttamente. L'esecuzione del programma è andata in timeout"
LLM Code Correct: "NO"
LLM Code Diagnosis: "The defect lies in FILE: serversync.c, function send_sinc, at line 53. The msgsnd call uses id_coda (derived from ok.id) as the target queue ID. However, the runtime output shows invalid queue IDs (e.g., negative values like -1375755542), indicating that the server's private queue ID is not correctly communicated to the client. This violates the protocol requiring the server to send a valid queue ID in the OK TO SEND message. The mismatch causes message delivery failures and incomplete client-server interactions, leading to the observed timeout."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies

a concrete defect in the student code, specifically the incorrect communication of the server's private queue ID to the client. This is supported by the diff, which shows that the student code uses ok.id instead of the correct ack_msg.server_queue_id from the solution. The observed timeout failure aligns with this defect, as it would cause message delivery issues. The diagnosis does not introduce unrelated or speculative issues and matches the visible differences in the diff."

Esempio timeout 2

Studente: "ariannaraiola"
Esercizio: "server_sincroni_multipli"
Commit: "64bec254540a1cfaf4118a1414c7f46ca5bb99bc"
Test feedback: "Esercizio server sincroni multipli. L'esercizio non è ancora stato svolto correttamente. L'esecuzione del programma è andata in timeout"
LLM Code Correct: "NO"
LLM Code Diagnosis: "The defect lies in FILE: serversync.c, function send_sinc, specifically in the handling of the OK TO SEND response. The msgrcv call at line 42 incorrectly receives the response on the req_id queue instead of the ok_id queue. This violates the protocol requiring OK TO SEND messages to be received from the ok_id queue. Consequently, the id_coda remains uninitialized, leading to the invalid queue ID (id_coda=0) in subsequent msgsnd calls, causing the "Invalid argument" errors and incomplete execution. This is a protocol-matching defect."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis identifies a protocol-matching defect in the handling of the OK TO SEND response, specifically pointing to the msgrcv call receiving from the wrong queue (req_id instead of ok_id). This is consistent with the observed timeout failure and aligns with the diff, which shows the student code present while the solution code is missing, implying the diagnosis is based on the student's implementation. The diagnosis does not introduce unrelated defects or unsupported speculation."

Esempio timeout 3

Studente: “talpinum”
Esercizio: “registro_distribuito”
Commit: “78b27e088e1ed536b349fbdbe904eaa80adbb468”
Test feedback: “Esercizio registro distribuito. L’esercizio non è ancora stato svolto correttamente. L’esecuzione del programma è andata in timeout”
LLM Code Correct: “NO”
LLM Code Diagnosis: “The timeout and incomplete termination are caused by the missing exit(0) in the registro function in FILE: registro.c at line 118. While the EXIT message is sent to the servers, the registro process itself does not terminate after completing its exit handling. This omission causes the parent process to wait indefinitely for the registro process to terminate, leading to the observed timeout. The missing termination violates the cleanup/termination requirement explicitly stated in the exercise description.”
Judge Code Correct: “YES”
Judge Code Motivation: “The diagnosis correctly identifies the missing exit(0) in the registro function as the cause of the timeout. The diff shows that the solution includes an exit(0) in the registro function’s parent process termination logic, while the student code does not. This omission aligns with the diagnosis’s explanation of the parent process waiting indefinitely for the registro process to terminate, leading to the timeout. The cited file, function, and mechanism are all supported by the diff.”

4.2 Esempi di risultati LLM nell’analisi di IPC leak

Esempio ipc leak 1

Studente: “Gio-34”
Esercizio: “load_balancing”
Commit: “bf306d64aaa33b0ed8fb01ed7486a6aa8d2485b4”
Test feedback: “Esercizio load balancing. L’esercizio non è ancora stato svolto correttamente. È necessario deallocare le risorse IPC al termine della esecuzione”

LLM Code Correct: “NO”
LLM Code Diagnosis: “The defect lies in FILE: procedure.c, function Balancer, at line 76. The msgsnd(...) call sends messages to the server queues without including a unique message type or discriminator. Since all servers use msgrcv(...) with a generic type of 0 (line 115 in function Server), there is no mechanism to ensure proper message routing or separation. This violates the protocol-matching requirement, leading to mismatched counts and incomplete message handling as observed in the runtime output. Each server should receive only its intended messages, but the lack of a type-based protocol causes misrouting.”
Judge Code Correct: “YES”
Judge Code Motivation: “The diagnosis correctly identifies a defect in the msgsnd and msgrcv calls in procedure.c, specifically the lack of a unique message type or discriminator. The diff shows that the student code does not use a request structure or a REQUEST_SIZE macro, and the msgsnd and msgrcv calls in the student code lack the necessary type-based protocol for proper message routing. This aligns with the claimed defect mechanism of mismatched message handling due to protocol violations.”

Esempio ipc leak 2

Studente: “ManuelParasmo”
Esercizio: “grafo_delle_dipendenze”
Commit: “f2386ca53eacb87f8914e4f814554339165f2f2a”
Test feedback: “Esercizio grafo delle dipendenze. L’esercizio non è ancora stato svolto correttamente. È necessario deallocare le risorse IPC al termine della esecuzione”
LLM Code Correct: “NO”
LLM Code Diagnosis: “The "stack smashing detected" error is caused by incorrect array indexing in FILE: p1.c, lines 71-73 and 87-91. The vettore array in the struct operandi is indexed starting from 1 instead of 0, leading to out-of-bounds writes during message preparation for msgsnd. This corrupts the stack and causes the runtime failure. The correct indices should be vettore[0] and vettore[1] for two-element messages and vettore[0] to vettore[3] for four-element messages. This violates memory safety and di-

srupts the protocol."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the issue of incorrect array indexing in vettore (e.g., vettore[1] instead of vettore[0]) in p1.c lines 71-73, 87-91, and 108-109. The diff shows that the solution uses operators[0] to operators[3] for proper indexing, while the student code uses vettore[1] to vettore[4], which aligns with the described out-of-bounds writes causing stack corruption. The diagnosis is specific, matches the visible differences, and is supported by the evidence."

Esempio ipc leak 3

Studente: "GaeVentura"
Esercizio: "server_sincroni_multipli"
Commit: "36ef25390ab768aed769a5a8757d10eb0fa67f83"
Test feedback: "Esercizio server sincroni multipli. L'esercizio non è ancora stato svolto correttamente. È necessario deallocare le risorse IPC al termine della esecuzione"
LLM Code Correct: "NO"
LLM Code Diagnosis: "The code in FILE: main.c, lines 99-100, incorrectly uses msgctl with ok_key and req_key instead of the queue IDs ok_id and req_id for deallocating the message queues. This causes the queues to remain undeleted, leading to the observed ipc_leak runtime failure. The correct parameters for msgctl should be the queue IDs returned by msgget, not the keys used to generate them. This violates the cleanup/termination requirement of properly releasing IPC resources."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis identifies a concrete defect (incorrect use of msgctl with keys instead of queue IDs) that directly aligns with the observed ipc_leak failure and is consistent with the cleanup/termination requirement. The diff does not contradict this mechanism, and the diagnosis does not introduce unrelated or speculative issues."

4.3 Esempi di risultati LLM nell'analisi di dynamic failure

Esempio dynamic failure 1

Studente: "FraLip"
Esercizio: "server_sincroni_multipli"
Commit: "d8e2e996a33154d295b76ea4806ecf41c4a9f99c"
Test feedback: "Esercizio server sincroni multipli. L'esercizio non è ancora stato svolto correttamente. L'esecuzione non è corretta: Il server riceve un messaggio con campo tipo non coerente con il PID del client, oppure con un valore incoerente rispetto a quello inviato dal client"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output indicates multiple errors in creating message queues ("Errore creazione coda messaggi" and "Errore creazione coda ok-to-send"). These errors prevent the initialization of the required communication infrastructure, making it impossible for the client-server protocol to proceed. The expected interactions, such as REQUEST TO SEND and OK TO SEND exchanges, are entirely missing. The trace does not show any concrete evidence of successful message exchanges or protocol completion."
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: server.c, line 42, the server process attempts to create a private message queue using key_t msg_key = IPC_PRIVATE and msgget(msg_key, IPC_CREAT | 0644). However, the msgget call for the private queue fails, as indicated by the runtime error "Errore creazione coda messaggi: No such file or directory." This failure prevents the server from initializing its private queue, which is critical for the client-server communication protocol. The issue likely stems from incorrect permissions or a missing prerequisite setup for the message queue creation. This defect blocks the protocol's execution."
Judge Output Correct: "NO"
Judge Output Motivation: "The diagnosis identifies errors in message queue creation and missing interactions, which are unrelated to the ground truth issue of the server receiving a message with an incoherent type field. The diagnosis describes a different

category of failure than the ground truth."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis claims the failure of msgget for the private queue due to incorrect permissions or missing setup. The diff shows a mismatch in permissions (0644 vs. 0664) for the msgget call in server.c, which aligns with the claimed defect mechanism. The runtime error and feedback about incorrect message handling are consistent with this issue."

Esempio dynamic failure 2

Studente: "ragdolla"
Esercizio: "load_balancing"
Commit: "8c94800da85b56863fdd5d9674ef44302c580258"
Test feedback: "Esercizio load balancing. L'esercizio non è ancora stato svolto correttamente. L'esecuzione non è corretta: Un server riceve un messaggio con un PID client che non coincide con nessuno dei client noti al balancer. Il numero di messaggi inviati dai client non coincide con quello ricevuto dai server"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output shows mismatched counts and ordering in the load balancing protocol. Specifically, the Balancer forwards 18 messages (6 per client, 2 clients, 3 servers), but the Server logs show inconsistent reception counts: Server 196039 receives 4 messages, Server 196040 receives 4 messages, and Server 196041 receives 4 messages, totaling 12 instead of 18. Additionally, the round-robin distribution is not consistently followed, as some servers receive fewer messages than expected. This violates the expected even distribution and completion of all client requests."
LLM Code Correct: "NO"
LLM Code Diagnosis: "The defect lies in FILE: procedure.c, function Balancer, at line 65 where msgrcv(...) is called without specifying a message type. This causes the Balancer to receive messages in an arbitrary order from the shared queue, violating the round-robin protocol. The lack of a specific message type selector leads to mismatched counts and uneven distribution of messages to servers, as seen in the runtime output where servers

receive fewer messages than expected. The exercise requires even distribution, which is broken due to this protocol-matching defect."
Judge Output Correct: "NO"
Judge Output Motivation: "The diagnosis identifies mismatched counts in message distribution and mentions a failure in round-robin distribution, which partially aligns with the ground truth's mention of mismatched message counts. However, the ground truth specifies an additional issue: a server receiving a message with an unknown client PID, which the diagnosis does not address. The diagnosis also does not explicitly confirm the mismatch in total messages sent versus received, focusing instead on distribution inconsistencies. Thus, it does not fully capture the essence of the ground truth failure."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the defect in procedure.c, function Balancer, at line 65, where msgrcv is called without specifying a message type. This matches the visible diff, which shows the student code using msgrcv with a generic type (0) instead of the specific type used in the solution. The diagnosis aligns with the observed feedback about mismatched counts and incorrect message handling, and no unrelated or speculative issues are mentioned."

Esempio dynamic failure 3

Studente: "MarkCer16"
Esercizio: "registro_distribuito"
Commit: "f301dec6db8c64bea3e2705205f3020bc9f42eaa"
Test feedback: "Esercizio registro distribuito. L'esercizio non è ancora stato svolto correttamente. L'esecuzione non è corretta: Il numero di richieste SERVICE inviate dal client non coincide con quello ricevuto dai server"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output shows multiple concrete mismatches. Clients receive responses with id_coda=0 instead of the expected queue IDs (e.g., 327698 or 327697), leading to invalid SERVICE messages. Errors such as "Errore send client(service):

Invalid argument" confirm that the SERVICE messages are malformed or routed incorrectly. Additionally, the EXIT messages sent by the registry reference servers 0 and 1, but the servers are identified as 1 and 2, indicating a mismatch in server identifiers. These issues violate the protocol's coherence and pairing requirements."
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: registro.c, function registro, the response message for a QUERY request (lines 73-82) does not correctly populate the id_coda field of the messaggio_registro structure before sending it to the client. This results in clients receiving responses with id_coda=0 instead of the expected queue IDs (e.g., 327697 or 327698). Consequently, clients send malformed SERVICE messages to invalid queue IDs, causing "Errore send client(service): Invalid argument" and breaking the protocol's coherence. This violates the protocol-matching requirement for correct request-response pairing."
Judge Output Correct: "YES"
Judge Output Motivation: "The diagnosis identifies specific mismatches in the output, such as incorrect id_coda values and server identifier mismatches, which align with the ground truth's statement that the execution is incorrect due to mismatched SERVICE requests. These details enrich the generic ground truth without contradicting it, as they describe the same category of failure (protocol coherence and pairing issues)."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies that the id_coda field of the messaggio_registro structure is not populated in the student's code (lines 73-82), while the solution explicitly assigns id_reg->server_queue = id_coda (line 72). This omission directly aligns with the described defect mechanism of clients receiving responses with id_coda=0, causing protocol coherence issues. The diff supports the diagnosis concretely."

4.4 Esempi di risultati LLM nell'analisi di static failure

Esempio static failure 1

Studente: "andreatrimarchi"
Esercizio: "registro_distribuito"
Commit: "0c3494ca64b2e53122c91c375678febe38154278"
Test feedback: "Esercizio registro distribuito. L'esercizio non è ancora stato svolto correttamente. Anche se il programma esegue, sono stati riscontrati i seguenti difetti all'interno del codice (server.c)"
Static warnings: ["file": "server.c", "line": 38, "message": "È necessario che i processi server utilizzino code distinte. Se i processi chiamano tutti ftok() con gli stessi parametri, ottengono la stessa chiave e quindi la stessa coda. Il modo suggerito di risolvere l'esercizio è utilizzare IPC_PRIVATE (le code non avranno una chiave, e msgget restituirà uno ID distinto)."]
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output shows mismatched message values and counts. Server 2 logs receiving SERVICE messages with values 6, 2, 8, and 9, but the clients only sent values 6, 9, and 9 to Server 2. Additionally, the registry sends EXIT messages to "server 0" and "server 1," but no "server 0" exists in the trace. These inconsistencies indicate protocol violations in message pairing and actor identity."
LLM Code Correct: "NO"
LLM Code Diagnosis: "FILE: server.c, function server, line 12: The server's private message queue is created with a fixed key using ftok(".", 'e'). This approach risks key collisions if multiple servers are started on the same system or if the key is reused elsewhere. A unique key generation mechanism is required to ensure proper lifetime and ownership of the message queue for each server."
Judge Output Correct: "NO"
Judge Output Motivation: "The ground truth states that the visible execution/output is correct, but the diagnosis claims mismatched message values, counts, and protocol violations, which contradicts the ground truth. The diagnosis invents problems not

supported by the ground truth."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the same defect mechanism as the ground truth. Both highlight the issue of key collisions caused by using ftok() with fixed parameters, and the need for a unique key generation mechanism (e.g., IPC_PRIVATE) to ensure distinct message queues for each server. The diagnosis provides a detailed explanation of the root cause and solution, which aligns with the ground truth warning."

Esempio static failure 2

Studente: "TheDarkBug"
Esercizio: "registro_distribuito"
Commit: "36fe6e3a3217ad4c08f0d844afac792b2466f959"
Test feedback: "Esercizio registro distribuito. L'esercizio non è ancora stato svolto correttamente. Anche se il programma esegue, sono stati riscontrati i seguenti difetti all'interno del codice (server.c)"
Static warnings: ["file": "server.c", "line": 35, "message": "È necessario che i processi server utilizzino code distinte. Se i processi chiamano tutti ftok() con gli stessi parametri, ottengono la stessa chiave e quindi la stessa coda. Il modo suggerito di risolvere l'esercizio è utilizzare IPC_PRIVATE (le code non avranno una chiave, e msgget restituirà uno ID distinto)."]
LLM Output Correct: "YES"
LLM Output Diagnosis: "No concrete output mismatch is visible. The trace structure is consistent with the exercise protocol. All expected actors (servers, clients, and the registry) are present, and their interactions (BIND, QUERY, SERVICE, and EXIT) follow the described protocol. Message counts, pairings, and orderings are coherent, and at least one complete request/-reply cycle was spot-checked and verified as correct. All phases, including initialization, service exchanges, and termination, are accounted for."
LLM Code Correct: "NO"
LLM Code Diagnosis: "FILE: server.c, function server, line 12: The msgget call uses id_server as the key for creating the

server's private message queue. However, id_server is not guaranteed to be unique across different servers, leading to potential key collisions. This violates the lifetime/ownership constraint, as multiple servers could inadvertently share or overwrite the same message queue, causing undefined behavior."
Judge Output Correct: "YES"
Judge Output Motivation: "The diagnosis correctly states that no concrete output mismatch is visible and aligns with the ground truth, which confirms the output is correct. It does not invent any additional problems or contradict the ground truth."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the same defect mechanism as the ground truth. Both point out that the use of non-unique keys (e.g., id_server or ftok() with identical parameters) can lead to key collisions and shared message queues, violating the requirement for distinct queues per server. The diagnosis also aligns with the ground truth's suggested resolution of using IPC_PRIVATE to ensure unique queue IDs. The causal chain and defect family are consistent."

Esempio static failure 3

Studente: "CerottinoPasquale"
Esercizio: "registro_distribuito"
Commit: "e5285df6c3f232026dc2a08461339205964149 67"
Test feedback: "Esercizio registro distribuito. L'esercizio non è ancora stato svolto correttamente. Anche se il programma esegue, sono stati riscontrati i seguenti difetti all'interno del codice (server.c)"
Static warnings: ["file": "server.c", "line": 43, "message": "È necessario che i processi server utilizzino code distinte. Se i processi chiamano tutti ftok() con gli stessi parametri, ottengono la stessa chiave e quindi la stessa coda. Il modo suggerito di risolvere l'esercizio è utilizzare IPC_PRIVATE (le code non avranno una chiave, e msgget restituirà uno ID distinto)."]
LLM Output Correct: "NO"
LLM Output Diagnosis: "The trace shows "Registro: Invio messaggio EXIT a server 0" instead of "Registro: Invio messaggio EXIT a server 1 and server 2". This indicates a mismatch in the

server ID for the exit message. Additionally, the server IDs in the exit messages received by the servers are inconsistent with the IDs used throughout the trace. This is a concrete mismatch in the protocol."
LLM Code Correct: "NO"
LLM Code Diagnosis: "FILE: server.c, function server, line 13: The msgget call uses id_server directly as the key for creating the server's private message queue. This violates the lifetime/ownership constraint because id_server is not guaranteed to be a unique or valid key for msgget. This can lead to conflicts or incorrect queue creation, as id_server values (1 or 2) are reused across servers. A unique key generation mechanism, such as ftok, should be used to ensure proper queue ownership and avoid collisions."
Judge Output Correct: "NO"
Judge Output Motivation: "The ground truth explicitly states that the visible execution/output is correct, while the diagnosis claims a concrete mismatch in server IDs and protocol behavior. This directly contradicts the ground truth, which confirms correctness. Therefore, the diagnosis does not align with the ground truth."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis identifies the same underlying defect mechanism as the ground truth warning: the improper use of keys for message queues leading to potential conflicts. Both the diagnosis and the warning suggest using a unique key generation mechanism (e.g., ftok or IPC_PRIVATE) to resolve the issue. The diagnosis does not invent a different defect family and aligns with the ground truth's concern about queue ownership and collisions."

4.5 Esempi di risultati LLM nell'analisi di correct

Esempio correct 1

Studente: "luke-the-coder"
Esercizio: "load_balancing"
Commit: "075b2a297172d7775a87d2fa1026915f237af77b"

Test feedback: "Esercizio load balancing.Congratulazioni, l'esercizio compila ed esegue correttamente, e non è stato riscontrato nessuno degli errori frequenti nel codice"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output shows a mismatch in message distribution by the Balancer. Specifically, Client 332539 sends 6 messages, but the Balancer only processes 5 of them. Similarly, Client 332540 sends 6 messages, but the Balancer processes 7. This results in an incorrect total count of messages handled by the Balancer (12 sent vs. 12+1 processed). Additionally, the distribution to servers is uneven, with Server 1 receiving 4 messages, Server 2 receiving 4, and Server 3 receiving 4, which does not align with the expected round-robin protocol."
LLM Code Correct: "YES"
LLM Code Diagnosis: "No concrete correctness concrete defect is visible from the provided code and runtime context. The program adheres to the explicit requirements of the exercise, including the creation of processes, message queues, and the round-robin distribution of messages by the balancer. The observed output confirms that all clients, the balancer, and servers perform their roles correctly, with no missing or truncated protocol phases. All processes terminate as expected, and the message queues are properly removed."
Judge Output Correct: "NO"
Judge Output Motivation: "The ground truth explicitly states that the visible execution/output is correct, while the diagnosis claims multiple mismatches in message distribution and protocol adherence. This directly contradicts the ground truth, which confirms no issues in the output."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly states that no concrete correctness defect is visible, and the observed feedback confirms the program's correctness. The diff does not show any explicit correctness violations, and the differences are superficial (e.g., variable names, formatting, or minor structural variations) without impacting functionality. The diagnosis aligns with the rules and observed feedback."

Esempio correct 2

Studente: “itsdom1”
Esercizio: “registro_distribuito”
Commit: “976f7810e83565613c0ec0e0dce67696474f32a7”
Test feedback: “Esercizio registro distribuito.Congratulazioni, l’esercizio compila ed esegue correttamente, e non è stato riscontrato nessuno degli errori frequenti nel codice”
LLM Output Correct: “YES”
LLM Output Diagnosis: “No concrete output mismatch is visible. The trace structure is consistent with the exercise protocol. All expected phases are present: server BIND messages are sent and registered by the registry, clients send QUERY messages and receive correct responses, SERVICE messages are sent to the appropriate servers, and EXIT messages are handled correctly. Message counts, pairings, and ordering are coherent, and no visible value or arithmetic errors are detected. The overall structure aligns with the described protocol.”
LLM Code Correct: “NO”
LLM Code Diagnosis: “In registro.c, the constant "BLIND" is used instead of "BIND" in the condition if(msg_reg.type==BLIND) (line 27). This causes the server’s BIND messages to be misinterpreted, as the correct constant "BIND" is not defined or used. This results in a protocol violation where server registration may fail, despite the output showing successful execution.”
Judge Output Correct: “YES”
Judge Output Motivation: “The diagnosis correctly states that no concrete output mismatch is visible and aligns with the ground truth, which confirms the output is correct. It does not invent any additional problems or contradict the ground truth.”
Judge Code Correct: “YES”
Judge Code Motivation: “The diagnosis correctly identifies the use of "BLIND" instead of "BIND" in the condition if(msg_reg.type==BLIND) (line 27), which is visible in the diff and matches the described defect mechanism. The solution uses id_reg->type == BIND, confirming the mismatch. The defect aligns with a protocol violation as described, and the feedback does not contradict this specific issue.”

Esempio correct 3

Studente: "Beccio-san"
Esercizio: "grafo_delle_dipendenze"
Commit: "55273704c6faa4b520e87019938c7ecbdd6f9652"
Test feedback: "Esercizio grafo delle dipendenze.Congratulazioni, l'esercizio compila ed esegue correttamente, e non è stato riscontrato nessuno degli errori frequenti nel codice"
LLM Output Correct: "YES"
LLM Output Diagnosis: "No concrete output mismatch is visible. The trace structure is consistent with the exercise protocol. All expected processes (P1 to P6) are present, and their interactions (sending and receiving messages, computing results) follow the described dependency graph. Operand values and results are coherent with the formula y = (a*b + (c+d)*(e+f) + g*h) across all iterations. Message counts, pairings, and ordering are correct, and all processes terminate properly. No missing phases or invalid computations are observed."
LLM Code Correct: "YES"
LLM Code Diagnosis: "No concrete correctness concrete defect is visible from the provided code and runtime context. The execution output shows all processes (P1 to P6) correctly exchanging messages, performing computations, and terminating as expected. The results match the described dependency graph and formula requirements. All message queues are properly created, used, and removed, and the processes handle their respective tasks without observable protocol violations or incomplete phases."
Judge Output Correct: "YES"
Judge Output Motivation: "The diagnosis correctly states that no concrete correctness defects are visible in the provided code and runtime context. The observed feedback confirms the code compiles and executes correctly without errors, and the diff does not show any explicit correctness violations. The structural differences (presence of files in the student submission vs. their absence in the solution) do not constitute a correctness defect, as the reference solution is not canonical."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly states that there are no concrete correctness defects visible in the provided

code and runtime context. The observed feedback confirms the code compiles and executes correctly without errors, and the diff does not show any explicit correctness violations. The structural differences (presence of files in the student submission vs. their absence in the solution) do not constitute a correctness defect, as the reference solution is not canonical."

5 Esempi analisi LLM esercitazione 5

5.1 Esempi di risultati LLM nell'analisi di timeout

Esempio timeout 1

Studente: "TheDarkBug"
Esercizio: "server_aggregatore_thread"
Commit: "9e6483ee5d3518b2b8d90b3cb4ec68f97337ef16"
Test feedback: "Esercizio server aggregatore con thread. L'esercizio non è ancora stato svolto correttamente. L'esecuzione del programma è andata in timeout"
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: aggregatore.c, function lettura (lines 77-89), the reader-writer synchronization policy is violated. Specifically, the condition if (m->reader_count– == 0) (line 86) incorrectly decrements reader_count before checking if it is zero. This can cause a race condition where multiple readers decrement the count simultaneously, leading to incorrect signaling of the condition variable m->cond. This violates the reader-writer invariant, as readers must not interfere with each other's access to shared state."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a synchronization issue in the lettura function, specifically the decrement of reader_count before checking its value, which is visible in the diff (line 86 in the student code). The solution code handles this correctly by decrementing num_lettori after the critical section and signaling only when it reaches zero. This aligns with the claimed defect mechanism and the observed timeout feedback,

which is consistent with a race condition or synchronization error."

Esempio timeout 2

Studente: "leonardomassa"
Esercizio: "un_primo_esempio_di_server_multithread"
Commit: "021b80c1d78c07d743c50fbf2496d4c5fcdee71c"
Test feedback: "Esercizio primo esempio di server multithread. L'esercizio non è ancora stato svolto correttamente. L'esecuzione del programma è andata in timeout"
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: Server.c, function server, the pthread_create call at line 69 creates threads without any subsequent pthread_join or equivalent synchronization mechanism in the main server loop. This omission risks premature termination of the server process before all worker threads complete their tasks, violating the exercise's requirement for proper cleanup and termination. The parent process does not wait for the threads, and no alternative mechanism ensures their completion before the server exits. This defect directly impacts the lifetime and termination policy of the server."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the absence of pthread_join or equivalent synchronization in the student's Server.c implementation, specifically in the pthread_create call at line 69. This omission is visible in the diff, where the student code lacks any mechanism to ensure proper thread cleanup or synchronization, unlike the solution. The defect aligns with the observed timeout feedback, as improper thread management can lead to premature termination or hanging behavior. The diagnosis is concrete and supported by the diff."

Esempio timeout 3

Studente: "barigio"
Esercizio: "server_aggregatore_thread"
Commit: "dd0d3097d8d04d5ee0ab9def65a7ace9b1a3aad0"
Test feedback: "Esercizio server aggregatore con thread. L'e-

sercizio non è ancora stato svolto correttamente. L'esecuzione del programma è andata in timeout"
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: aggregatore.c, the param struct is shared across all threads (lines 52-90), but its id_coda_lett field is overwritten in the loop at line 67 before each reader thread is created. This causes all reader threads to use the same id_coda_lett value, leading to incorrect message forwarding to collectors. This violates the protocol requirement of distinct message queues for each collector and explains the observed timeout and mismatched message counts in the runtime output."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis identifies a concrete issue with the param struct being shared across threads and its id_coda_lett field being overwritten in a loop, which is consistent with the observed timeout and mismatched message counts. This mechanism is plausible for the failure and aligns with the rules for timeout failures. The diff does not contradict this diagnosis, and the feedback supports the described defect."

5.2 Esempi di risultati LLM nell'analisi di IPC leak

Esempio IPC leak 1

Studente: "MarcoDani"
Esercizio: "un_primo_esempio_di_server_multithread"
Commit: "df9c1facf34f753d60e04314f2288f60e04515d7"
Test feedback: "Esercizio primo esempio di server multithread. L'esercizio non è ancora stato svolto correttamente. È necessario deallocare le risorse IPC al termine della esecuzione"
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: Server.c, function Prodotto, the msgsnd(...) call at line 105 sends a response message without setting the risp.risposta field. This omission causes the client to receive uninitialized or incorrect values, as seen in the runtime output where negative or invalid results are received. The defect

violates the protocol requirement to send the computed product back to the client. The missing assignment to risp.risposta directly explains the observed incorrect response values."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies that the risp.risposta field is not set before the msgsnd call in the student's code, which directly aligns with the observed runtime issue of incorrect or uninitialized values being sent to the client. The diff shows that the solution initializes the risposta.risultato field, while the student's code does not set risp.risposta. This omission matches the claimed defect mechanism and the feedback about incorrect results."

Esempio IPC leak 2

Studente: "MarcoDani"
Esercizio: "remote_procedure_call"
Commit: "df9c1facf34f753d60e04314f2288f60e04515d7"
Test feedback: "Esercizio remote procedure call. L'esercizio non è ancora stato svolto correttamente. È necessario deallocare le risorse IPC al termine della esecuzione"
LLM Code Correct: "NO"
LLM Code Diagnosis: "The defect lies in FILE: server.c, function worker, specifically at lines 95-108. The risultato field in the response message is incorrectly set to a placeholder value (0) for PROD_SOMMA and PRODUZIONE requests, instead of propagating the computed or meaningful result. This violates the RPC protocol's requirement for immediate and correct replies to client requests. The correct values (e.g., 8) only appear later in CONSUMA replies, causing a mismatch in request/reply association and leading to the observed runtime failure."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a defect in the worker function of server.c related to the risultato field in the response message. The diff shows that the student code does not set the type field in the response message (msg_risposta.type = msg_richiesta.pid_client; is missing), which is critical for associating replies with requests. This aligns

with the diagnosis of a mismatch in request/reply association and incorrect propagation of results. The cited lines and mechanism are supported by the diff."

5.3 Esempi di risultati LLM nell'analisi di dynamic failure

Esempio dynamic failure 1

Studente: "leonardoricc2002"
Esercizio: "un_primo_esempio_di_server_multithread"
Commit: "7ae38b2d7eb48c13528d0f3391e6a102cd0ac6c1"
Test feedback: "Esercizio primo esempio di server multithread. L'esercizio non è ancora stato svolto correttamente. L'esecuzione non è corretta: Il valore del messaggio di risposta 0 del processo 55025 non corrisponde ad un messaggio di richiesta. Il numero di messaggi di richiesta e di risposta non corrisponde al totale atteso (15)"
LLM Code Correct: "YES"
LLM Code Diagnosis: "No concrete output mismatch is visible. The trace shows consistent request/reply interactions for all three clients, with each client sending 5 requests and receiving 5 replies. The computed product values in the replies match the input values in the corresponding requests. The server logs also confirm the correct computation and sending of results. The termination message is present, indicating proper completion of the protocol."
LLM Code Correct: "YES"
LLM Code Diagnosis: "No concrete correctness concrete defect is visible from the provided code and runtime context. The observed runtime output matches the expected behavior, with all clients sending 5 requests and receiving correct replies. The server correctly processes requests, uses a mutex for mutual exclusion on the response queue, and terminates upon receiving the special message {-1, -1}. No explicit violations of protocol, synchronization, or resource management are evident in the code."
Judge Output Correct: "NO"
Judge Output Motivation: "The diagnosis claims there is no

visible mismatch and that the execution is correct, but the ground truth explicitly states that the execution is incorrect due to mismatched request/response counts and a specific value mismatch. The diagnosis fails to capture the essence of the failure described in the ground truth."
Judge Code Correct: "NO"
Judge Code Motivation: "The diagnosis claims no concrete correctness defect is visible, but the observed test feedback explicitly reports mismatched request/response values and counts. The diff shows concrete differences in the struct field names (v1, v2 vs. val1, val2) and message handling logic (e.g., ric vs. richiesta), which directly align with the reported protocol mismatch. This contradicts the diagnosis's assertion of no visible defect."

Esempio dynamic failure 2

Studente: "iacnella"
Esercizio: "remote_procedure_call"
Commit: "c31d771174f19bf0e17c26691a55801677f4f839"
Test feedback: "Esercizio remote procedure call. L'esercizio non è ancora stato svolto correttamente. L'esecuzione non è corretta: Errore nel valore di PRODUCI CON SOMMA ricevuto dal server. Il numero totale di messaggi non corrisponde a quello richiesto dalla traccia"
LLM Code Correct: "NO"
LLM Code Diagnosis: "The immediate replies to PRODUCI_CON_SOMMA and PRODUCI requests consistently return a placeholder value (risultato=0), while the meaningful results (e.g., 12, 3, 7) appear only later during CONSUMA operations. This indicates a visible mismatch in the request/reply protocol, as the client-visible replies do not reflect the expected computed results at the time of the reply. The protocol requires correct request/reply association, which is violated here."
LLM Code Correct: "NO"
LLM Code Diagnosis: "In FILE: server.c, function worker, the condition for handling "CONSUMAZIONE" requests at line 124 is incorrect. The condition is written as else if(CONSUMAZIONE) instead of else if(req.type == CONSUMA-

ZIONE). This causes the worker to incorrectly process requests, as the condition does not properly check the message type. This violates the protocol-matching requirement, leading to potential misclassification of request types and incorrect behavior during message handling."
Judge Output Correct: "YES"
Judge Output Motivation: "The diagnosis identifies a mismatch in the request/reply protocol, specifically pointing out that the immediate replies to PRODUCI_CON_SOMMA and PRODUCI requests return placeholder values instead of the expected computed results. This aligns with the ground truth, which states there is an error in the value of PRODUCI_CON_SOMMA received from the server. Both highlight a failure in the correctness of the output values, making the diagnosis semantically aligned with the ground truth."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a concrete issue in server.c, function worker, where the condition for handling "CONSUMAZIONE" requests is improperly written as else if(CONSUMAZIONE) instead of else if(req.type == CONSUMAZIONE). This aligns with the visible diff, which shows mismatched handling of request types (req.type vs. msg_richiesta.type) and supports the protocol-matching defect mechanism. The feedback about incorrect behavior during message handling and mismatched message counts further corroborates this issue."

Esempio dynamic failure 3

Studente: "fabioauss-1"
Esercizio: "remote_procedure_call"
Commit: "65da47f44702d840d1b6d5e31992d06563801af4"
Test feedback: "Esercizio remote procedure call. L'esercizio non è ancora stato svolto correttamente. Anche se il programma esegue, sono stati riscontrati i seguenti difetti all'interno del codice (server.c)"
LLM Code Correct: "NO"
LLM Code Diagnosis: "The trace shows mismatched ordering

and pairing of operations. Specifically, a CONSUMA request is sent before the corresponding PRODUCI(5) request is processed, leading to ambiguous value flows. Additionally, the PRODUCI_CON_SOMMA operation's result (14) is consumed correctly, but subsequent PRODUCI and CONSUMA operations show inconsistent pairing and result propagation. The visible counts and ordering of requests and replies do not fully align with the expected protocol, and operand-to-result correspondence cannot be consistently spotchecked for all operations."
LLM Code Correct: "NO"
LLM Code Diagnosis: "FILE: server.c, function worker, lines 48-52: The dynamically allocated memory for the struct param object is not freed after being passed to the worker thread. This violates proper lifetime/ownership management, as the memory allocated with malloc is never released, leading to a memory leak. The code should ensure that the allocated memory is freed, either within the worker thread or after the thread has completed execution."
Judge Output Correct: "NO"
Judge Output Motivation: "The diagnosis claims mismatched ordering, pairing, and result propagation issues in the output, but the ground truth explicitly states that the visible execution/output is correct. This creates a direct contradiction, as the diagnosis invents problems not supported by the ground truth."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies a memory leak caused by the failure to free dynamically allocated memory for the struct param object, which matches the defect mechanism implied by the ground truth warning. The diagnosis stays within the same defect family and does not introduce unrelated issues."

5.4 Esempi di risultati LLM nell'analisi di static failure

Esempio static failure 1

Studente: "leonardoricc2002"
Esercizio: "remote_procedure_call"
Commit: "f66173d4cf97939ea983983e6ad5aa922881522b"
Test feedback: "Esercizio remote procedure call.Congratulazioni, l'esercizio compila ed esegue correttamente, e non è stato riscontrato nessuno degli errori frequenti nel codice"
Static warnings: ["file": "server.c", "line": 70, "message": "È richiesto che la struttura dati con i parametri del thread worker venga deallocata dallo heap, utilizzando free()."]
LLM Output Correct: "NO"
LLM Code Diagnosis: "FILE: server.c, function worker, line 92: The msgrcv call does not use selective receive based on the pid_client field, violating the exercise's requirement to use the client's PID as the message type for responses. This allows workers to process messages indiscriminately, potentially leading to incorrect routing of responses. The defect lies in the lack of a selective receive mechanism to match the pid_client field in the request."
Judge Code Correct: "NO"
Judge Code Motivation: "The diagnosis identifies a defect in message routing (lack of selective receive), which is unrelated to the ground truth warning about heap deallocation of the thread parameter structure. These are distinct defect families (message handling vs. memory management), and the diagnosis does not address the actual ground truth defect."

Esempio static failure 2

Studente: "P0l1702"
Esercizio: "server_aggregatore_thread"
Commit: "d533bb921aac1425291cb690ffada88e9c9407ca"
Test feedback: "Esercizio server aggregatore con thread. L'esercizio non è ancora stato svolto correttamente. Anche se il pro-

gramma esegue, sono stati riscontrati i seguenti difetti all'interno del codice (aggregatore.c)"
Static warnings: ["file": "aggregatore.c", "line": 167, "message": "È necessario che i lettori possano leggere contemporaneamente il dato condiviso. Si applichi qui l'algoritmo visto a lezione."]
LLM Output Correct: "NO"
LLM Code Diagnosis: "FILE: aggregatore.c, function lettura (lines 125-142): The reader-writer synchronization policy is violated. The lettura function increments and decrements num_lett around the read operation but does not prevent concurrent writers from modifying variabile during the read. The pthread_mutex_unlock at line 132 allows writers to proceed before the reader decrements num_lett at line 137, breaking the reader-writer policy where readers should read concurrently but writers must wait until all readers finish."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly identifies the reader-writer synchronization policy violation, which aligns with the ground truth warning about ensuring readers can read concurrently without interference from writers. The diagnosis provides detailed reasoning about the mechanism (e.g., pthread_mutex_unlock allowing writers to proceed prematurely), which is causally related to the defect described in the ground truth. The line number discrepancy does not invalidate the semantic match."

Esempio static failure 3

Studente: "GaeVentura"
Esercizio: "un_primo_esempio_di_server_multithread"
Commit: "b1f2db2b1482cd726c6f515ab8337ffdb96785d4"
Test feedback: "Esercizio primo esempio di server multithread. L'esercizio non è ancora stato svolto correttamente. Anche se il programma esegue, sono stati riscontrati i seguenti difetti all'interno del codice (Server.c)"
Static warnings: ["file": "Server.c", "line": 53, "message": "È richiesto che la struttura dati con i parametri del thread worker

venga deallocata dallo heap, utilizzando free()."]
LLM Output Correct: "YES"
LLM Code Diagnosis: "FILE: Server.c, function Prodotto, lines 117-123: The mutex used for mutual exclusion is incorrectly initialized. The mutex p->mutex is accessed in the worker thread, but it is not initialized in the main thread or dynamically allocated message structure. This violates the synchronization policy requiring proper mutex initialization before use, leading to undefined behavior during locking and unlocking operations. The correct mutex mutex declared globally in Server.c should be used instead."
Judge Code Correct: "NO"
Judge Code Motivation: "The ground truth identifies a defect related to memory management, specifically the requirement to deallocate the thread worker's parameter structure from the heap using free(). The diagnosis, however, focuses on a synchronization issue involving the incorrect initialization of a mutex and its use in the worker thread. These are entirely different defect families (memory management vs. synchronization), and the diagnosis does not address the ground truth defect."

5.5 Esempi di risultati LLM nell'analisi di correct

Esempio correct 1

Studente: "Mattia-Iengo"
Esercizio: "remote_procedure_call"
Commit: "b16724490cbe75c8054f044a5aabe9656c020002"
Test feedback: "Esercizio remote procedure call.Congratulazioni, l'esercizio compila ed esegue correttamente, e non è stato riscontrato nessuno degli errori frequenti nel codice"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output shows a "double free or corruption (fasttop)" error, indicating a memory management issue. This is a concrete protocol violation as it disrupts the expected behavior of the system. Additionally, there are inconsistencies in the reply values: a PRODUCI operation incorrectly

sends a result of 4 instead of 0, and the termination logs appear prematurely. These issues indicate mismatched counts, improper memory handling, and protocol coherence problems."
LLM Code Correct: "NO"
LLM Code Diagnosis: "The code in "server.c", function "worker", contains a memory management issue. The struct param *p is allocated once in the main thread but is freed inside each worker thread after processing requests (free(p) at line 143). This leads to multiple threads attempting to free the same memory, causing the observed "double free or corruption" error during runtime. This is an explicit correctness concrete defect that violates proper memory management."
Judge Output Correct: "NO"
Judge Output Motivation: "The diagnosis invents multiple output problems (memory management issue, incorrect result, premature termination logs) that contradict the ground truth, which states the output is correct. These claims do not align with the ground truth."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis claims a memory management issue in server.c, specifically that struct param *p is allocated once in the main thread but freed in each worker thread, leading to a "double free or corruption" error. However, the diff shows that the student code allocates struct param *p once in the main thread (line 38) and passes the same pointer to all threads (line 53). In contrast, the solution allocates a separate struct param *p for each thread (line 50). This difference directly supports the diagnosis mechanism. The feedback does not contradict this, as it is generic and does not address memory management issues explicitly."

Esempio correct 2

Studente: "Claudio0212"
Esercizio: "un_primo_esempio_di_server_multithread"
Commit: "b370e6e68b1e66dfd850b8eae2e08e2a5b48994f"
Test feedback: "Esercizio primo esempio di server multithread.Congratulazioni, l'esercizio compila ed esegue correttamente,

e non è stato riscontrato nessuno degli errori frequenti nel codice"
LLM Output Correct: "NO"
LLM Output Diagnosis: "The output contains repeated identical requests and responses for each client (e.g., (12, 17) always producing 204 for PID=2275262). This violates the exercise's requirement for random integer selection in each request. Additionally, the server's S̈ono Prodotto di Servermessages are inconsistent in count and do not align with the expected 15 total requests (5 per client). These issues indicate protocol violations in randomness and message handling."
LLM Code Correct: "YES"
LLM Code Diagnosis: "No concrete correctness concrete defect is visible from the provided code and runtime context. The execution output matches the explicit requirements of the exercise, including correct message exchange, processing, and synchronization using a mutex. All threads correctly handle mutual exclusion for the response queue, and the server terminates as expected upon receiving the termination message."
Judge Output Correct: "NO"
Judge Output Motivation: "The ground truth explicitly states that the visible execution/output is correct, while the diagnosis claims specific protocol violations (repeated requests/responses, randomness issues, and message count inconsistencies). These claims contradict the ground truth, which confirms correctness. Therefore, the diagnosis does not align with the ground truth."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly states that no concrete correctness defect is visible, and the observed feedback confirms the code compiles and executes correctly without errors. The diff shows structural differences (e.g., variable naming, struct field names) but no explicit correctness violations. The diagnosis aligns with the rules and does not invent unsupported defects."

Esempio correct 3

Studente: "andrewng47"
Esercizio: "un_primo_esempio_di_server_multithread"
Commit: "1694871f2f9c06bba45ba45d4852ed06f0e335d6"

Test feedback: "Esercizio primo esempio di server multithread.Congratulazioni, l'esercizio compila ed esegue correttamente, e non è stato riscontrato nessuno degli errori frequenti nel codice"
LLM Output Correct: "YES"
LLM Output Diagnosis: "No concrete output mismatch is visible. The trace structure is consistent with the exercise protocol. All expected actors (3 clients and 1 server) are present, and each client sends 5 requests and receives 5 replies. The server correctly computes and sends the product of the two integers for each request. The counts, pairing, and ordering of requests and replies are coherent, and at least one complete request/reply cycle was spot-checked and verified as correct. The termination phase is not explicitly shown but does not affect the correctness of the visible trace."
LLM Code Correct: "YES"
LLM Code Diagnosis: "No concrete correctness concrete defect is visible from the provided code and runtime context. The implementation adheres to the explicit requirements of the exercise, and the observed execution output matches the expected behavior. The use of mutex for mutual exclusion on the response queue is correctly implemented, and all message exchanges between clients and the server are consistent with the described protocol. The program terminates as expected upon receiving the termination message."
Judge Output Correct: "YES"
Judge Output Motivation: "The diagnosis correctly states that the visible execution/output is correct, aligns with the ground truth, and does not introduce any invented problems. It provides detailed verification consistent with the ground truth's confirmation of correctness."
Judge Code Correct: "YES"
Judge Code Motivation: "The diagnosis correctly states that the code is functionally correct, adheres to the requirements, and matches the expected behavior. The diff does not show any explicit correctness violations, and the observed feedback confirms the submission is correct. Differences in variable names and struct field names are superficial and do not constitute a correctness defect."